Practical –9

AIM: To perform univariate, bivariate and multivariate data analysis on a given dataset and interpret the results.

- **Objectives:**
  - To understand different types of data analysis.
  - To calculate descriptive statistics for individual variables.
  - To analyze relationships between two or more variables.
  - To interpret patterns, trends and relationships within the dataset.

- **Theory:** Data analysis is the process of examining and interpreting data to discover meaningful patterns and relationships.
  - Univariate Analysis: Studies a single variable using measures such as mean, median, mode, variance and standard deviation. Histograms and boxplots can be used to understand its distribution.
  - Bivariate Analysis: Examines the relationship between two variables using techniques such as scatterplots, correlation and cross-tabulation.
  - Multivariate Analysis: Studies three or more variables simultaneously to understand complex relationships and identify patterns within the data.

➔ These techniques help summarize data, identify relationships and support data-driven decision-making.

- **Procedure:**
  1. Import the required Python libraries.
  2. Load the dataset into a Pandas DataFrame.
  3. Select the relevant numerical and categorical attributes.
  4. Perform univariate analysis using descriptive statistics and appropriate plots.
  5. Perform bivariate analysis to examine relationships between pairs of variables.
  6. Perform multivariate analysis using suitable statistical techniques and visualizations.
  7. Interpret the results and draw conclusions.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, NumPy, Matplotlib and Seaborn libraries.
  - **Dataset:** Online Retail Transactions Dataset

- **Code:**

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from scipy import stats
from retail_data import load_retail

sns.set_theme(style="whitegrid")
df = load_retail()
num = ["Quantity", "UnitPrice", "Revenue"]
print(df.shape); print(df.head())

# ====================== UNIVARIATE ======================
print("\n--- Descriptive statistics ---")
print(df[num].describe().T)
print("\nSkewness:\n", df[num].skew())
print("\nKurtosis:\n", df[num].kurt())

q99 = df[num].quantile(0.99)
fig, ax = plt.subplots(2, 3, figsize=(15, 8))
for i, c in enumerate(num):
    x = df.loc[df[c] <= q99[c], c]
    sns.histplot(x, bins=50, kde=True, ax=ax[0, i]); ax[0,
i].set_title(f"Histogram of {c} (≤ 99th pct)")
    sns.boxplot(x=x, ax=ax[1, i]);                    ax[1,
i].set_title(f"Boxplot of {c} (≤ 99th pct)")
plt.tight_layout(); plt.show()
```

```
# categorical variables
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
df["Country"].value_counts().head(10).plot(kind="bar", ax=ax[0], title="Top 10
countries by transactions")
df["InvoiceDate"].dt.month.value_counts().sort_index().plot(kind="bar",
ax=ax[1], title="Transactions per month")
plt.tight_layout(); plt.show()
print("UK share of transactions: %.1f%%" % (100 * (df.Country == "United
Kingdom").mean()))

# ====================== BIVARIATE ======================
sm = df[(df.Quantity <= q99.Quantity) & (df.UnitPrice <=
q99.UnitPrice)].sample(5000, random_state=1)
plt.figure(figsize=(7, 5))
sns.scatterplot(data=sm, x="UnitPrice", y="Quantity", alpha=.4)
plt.title("Quantity vs UnitPrice"); plt.show()

print("\nPearson  (Quantity, UnitPrice):", stats.pearsonr(df.Quantity,
df.UnitPrice))
print("Spearman (Quantity, UnitPrice):", stats.spearmanr(df.Quantity,
df.UnitPrice))

fig, ax = plt.subplots(1, 3, figsize=(18, 5))
df.groupby("Country")["Revenue"].sum().nlargest(10).plot(kind="bar", ax=ax[0],
title="Top-10 countries by revenue")
df.groupby(df.InvoiceDate.dt.to_period("M"))["Revenue"].sum().plot(ax=ax[1],
marker="o", title="Monthly revenue")
df.groupby(df.InvoiceDate.dt.hour)["Revenue"].sum().plot(kind="bar", ax=ax[2],
title="Revenue by hour of day")
plt.tight_layout(); plt.show()

df["is_uk"] = np.where(df.Country == "United Kingdom", "UK", "Non-UK")
uk, non = df.loc[df.is_uk == "UK", "Revenue"], df.loc[df.is_uk == "Non-UK",
"Revenue"]
print("\nMean revenue/line  UK: %.2f | Non-UK: %.2f" % (uk.mean(),
non.mean()))
print("Mann-Whitney U (UK vs Non-UK revenue):", stats.mannwhitneyu(uk, non))
```

```
# ======================== MULTIVARIATE ========================
cust = (df.dropna(subset=["CustomerID"]).groupby("CustomerID")
          .agg(TotalQty=("Quantity", "sum"), TotalRevenue=("Revenue", "sum"),
               NumInvoices=("InvoiceNo", "nunique"),
AvgUnitPrice=("UnitPrice", "mean"),
               NumProducts=("StockCode", "nunique"), Country=("Country",
"first")))
cust["Region"] = np.where(cust.Country == "United Kingdom", "UK", "Non-UK")
feat = ["TotalQty", "TotalRevenue", "NumInvoices", "AvgUnitPrice",
"NumProducts"]
logc = np.log1p(cust[feat])

sns.pairplot(pd.concat([logc, cust["Region"]], axis=1), hue="Region",
corner=True,
             plot_kws={"alpha": .4, "s": 15})
plt.suptitle("Pair plot of customer features (log scale)", y=1.02); plt.show()

plt.figure(figsize=(7, 5))
sns.heatmap(logc.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap (customer level, log-scaled)"); plt.show()

# 3 variables together: weekday x hour -> revenue
pv = df.pivot_table(index=df.InvoiceDate.dt.day_name(),
columns=df.InvoiceDate.dt.hour,
                    values="Revenue", aggfunc="sum")
pv = pv.reindex(["Monday", "Tuesday", "Wednesday", "Thursday", "Friday",
"Sunday"])
plt.figure(figsize=(12, 4)); sns.heatmap(pv, cmap="YlOrRd");
plt.title("Revenue: weekday × hour"); plt.show()

# ======================== INTERPRETATION ========================
print("\n--- Interpretation ---")
for c in num:
    print(f"{c}: skew={df[c].skew():.1f} -> strongly right-skewed (few very
large values); "
          f"mean {df[c].mean():.2f} > median {df[c].median():.2f}")
```

```
print("Quantity & UnitPrice are weakly (negatively) related: cheap items are bought in bulk.")
print("Revenue is dominated by the UK; sales peak around midday and in Oct-Nov (pre-Christmas).")
print("Customer features (qty, revenue, invoices, products) are strongly positively correlated.")
```

- **Output:**

```
python .\practical_9\prac9.py
PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_9\prac9.py
(530104, 9)
  InvoiceNo StockCode                          Description  Quantity
InvoiceDate  UnitPrice  CustomerID         Country  Revenue
0    536365    85123A   WHITE HANGING HEART T-LIGHT HOLDER         6
2010-12-01 08:26:00       2.55     17850.0  United Kingdom    15.30
1    536365     71053                  WHITE METAL LANTERN         6
2010-12-01 08:26:00       3.39     17850.0  United Kingdom    20.34
2    536365    84406B       CREAM CUPID HEARTS COAT HANGER         8
2010-12-01 08:26:00       2.75     17850.0  United Kingdom    22.00
3    536365    84029G  KNITTED UNION FLAG HOT WATER BOTTLE         6
2010-12-01 08:26:00       3.39     17850.0  United Kingdom    20.34
4    536365    84029E       RED WOOLLY HOTTIE WHITE HEART.         6
2010-12-01 08:26:00       3.39     17850.0  United Kingdom    20.34

--- Descriptive statistics ---
              count       mean         std    min   25%   50%    75%
max
Quantity   530104.0  10.542037  155.524124  1.000  1.00  3.00  10.00
80995.00
UnitPrice  530104.0   3.907625   35.915681  0.001  1.25  2.08   4.13
13541.33
Revenue    530104.0  20.121871  270.356743  0.001  3.75  9.90  17.70
168469.60

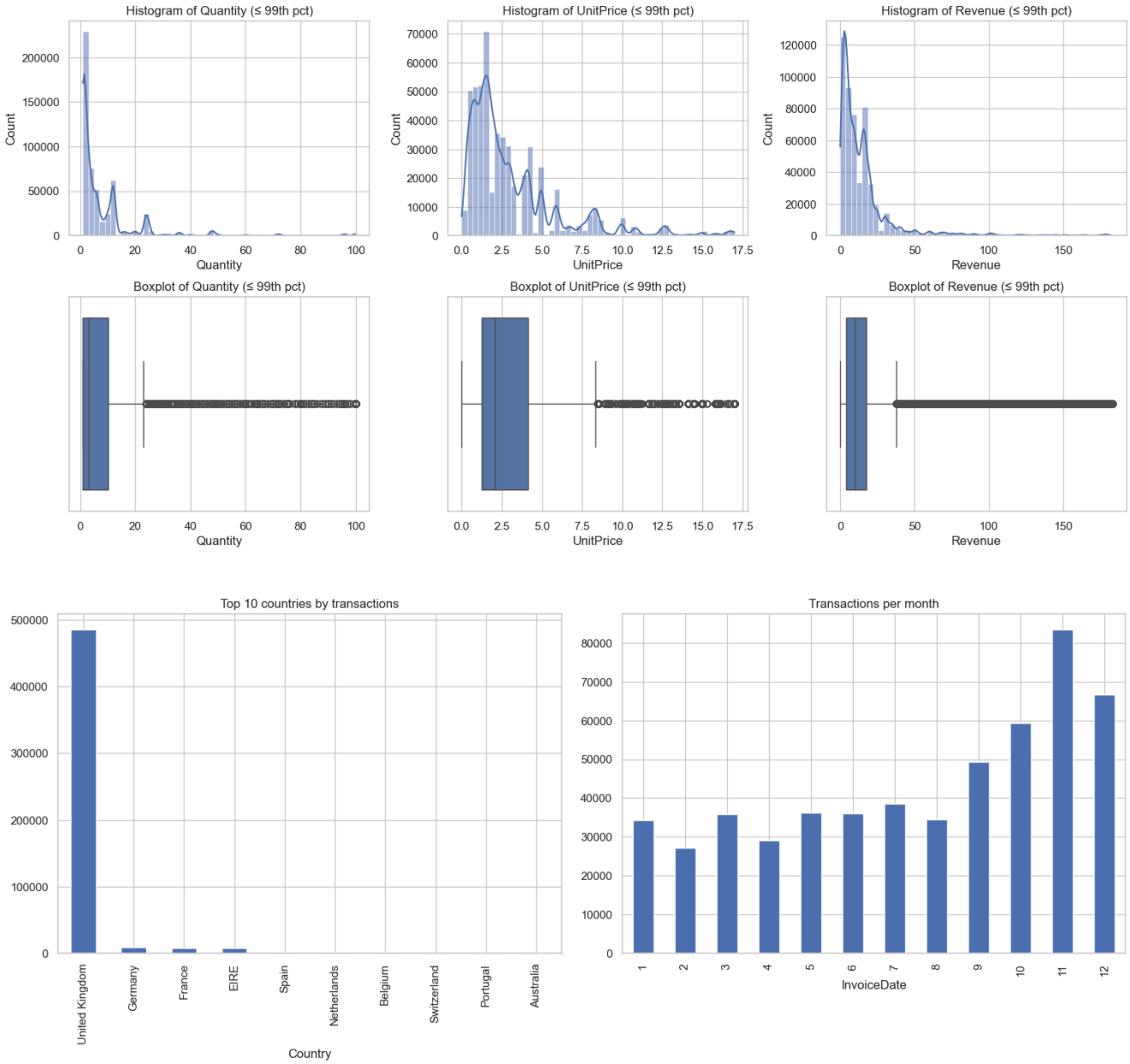
Skewness:
 Quantity     471.727716
UnitPrice    206.087555
Revenue      506.706012
dtype: float64
```

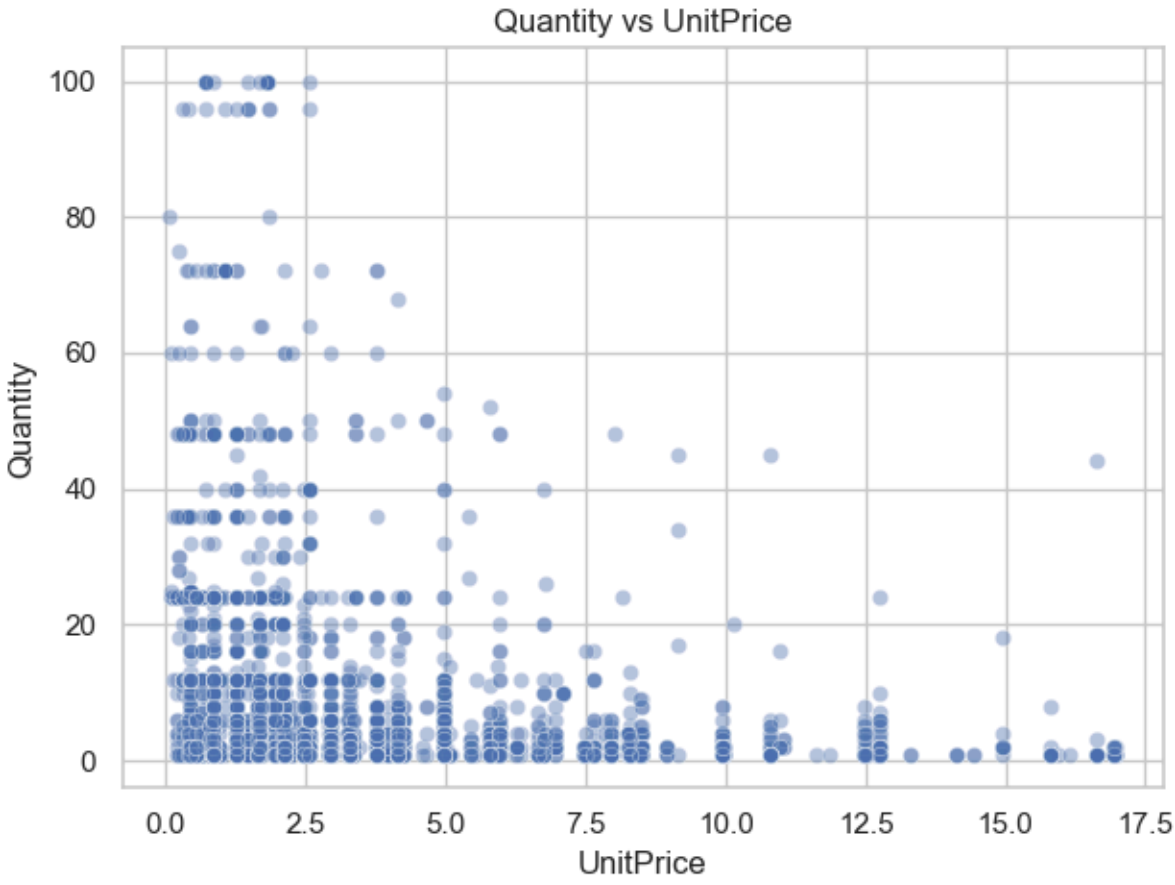
```
Kurtosis:
 Quantity     236462.342826
UnitPrice     62483.142715
Revenue      297651.661046
dtype: float64
UK share of transactions: 91.5%

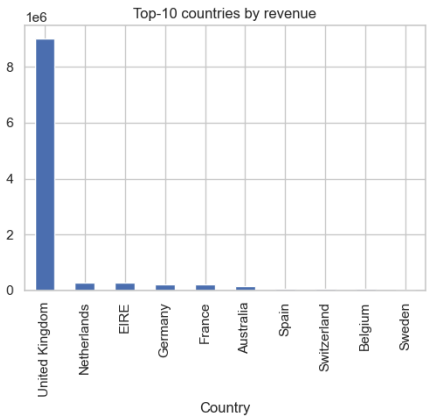
Pearson  (Quantity, UnitPrice):
PearsonRResult(statistic=np.float64(-0.003772542661636684),
pvalue=np.float64(0.006019353451639098))
Spearman (Quantity, UnitPrice):
SignificanceResult(statistic=np.float64(-0.4032372816383773),
pvalue=np.float64(0.0))

Mean revenue/line  UK: 18.60 | Non-UK: 36.49
Mann-Whitney U (UK vs Non-UK revenue):
MannwhitneyuResult(statistic=np.float64(5712161120.0),
pvalue=np.float64(0.0))

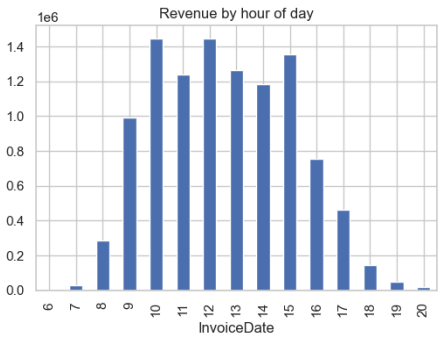
--- Interpretation ---
Quantity: skew=471.7 -> strongly right-skewed (few very large
values); mean 10.54 > median 3.00
UnitPrice: skew=206.1 -> strongly right-skewed (few very large
values); mean 3.91 > median 2.08
Revenue: skew=506.7 -> strongly right-skewed (few very large
values); mean 20.12 > median 9.90
Quantity & UnitPrice are weakly (negatively) related: cheap items
are bought in bulk.
Revenue is dominated by the UK; sales peak around midday and in
Oct-Nov (pre-Christmas).
Customer features (qty, revenue, invoices, products) are strongly
positively correlated.
```

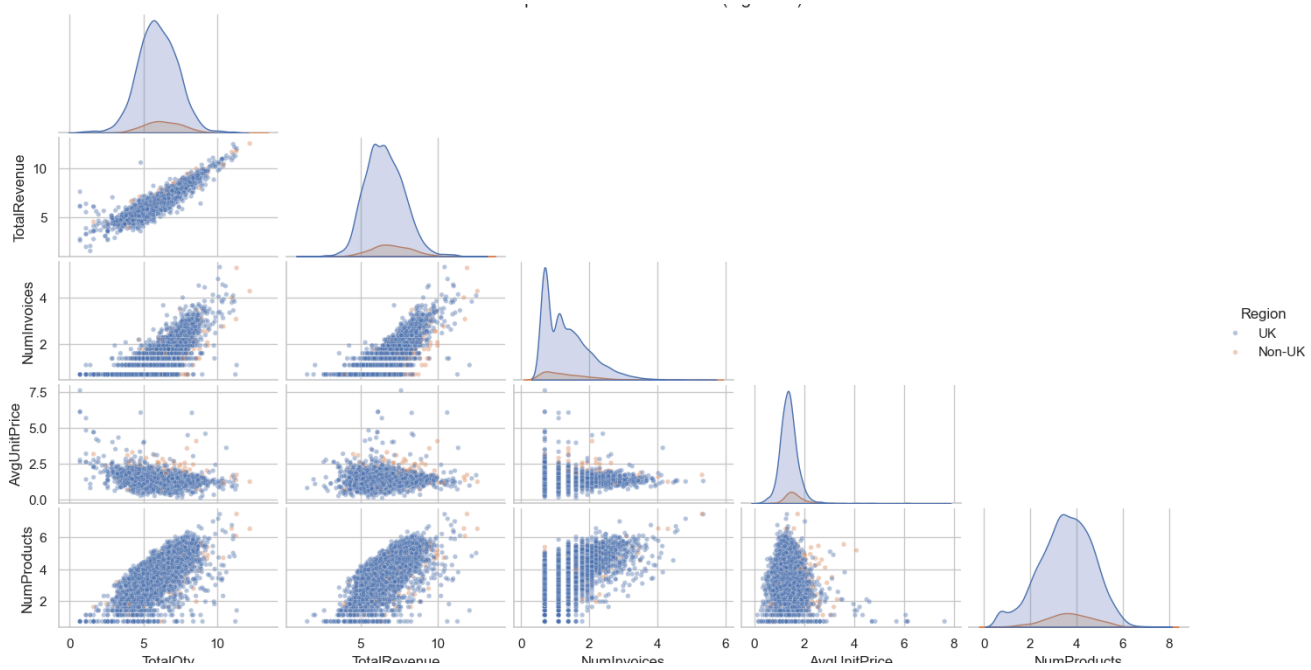

Histogram of Quantity (≤ 99th pct)
Count
Quantity
Histogram of UnitPrice (≤ 99th pct)
Count
UnitPrice
Histogram of Revenue (≤ 99th pct)
Count
Revenue
Boxplot of Quantity (≤ 99th pct)
Quantity
Boxplot of UnitPrice (≤ 99th pct)
UnitPrice
Boxplot of Revenue (≤ 99th pct)
Revenue
Top 10 countries by transactions
United Kingdom
Germany
France
EIRE
Spain
Netherlands
Belgium
Switzerland
Portugal
Australia
Country
Transactions per month
InvoiceDate

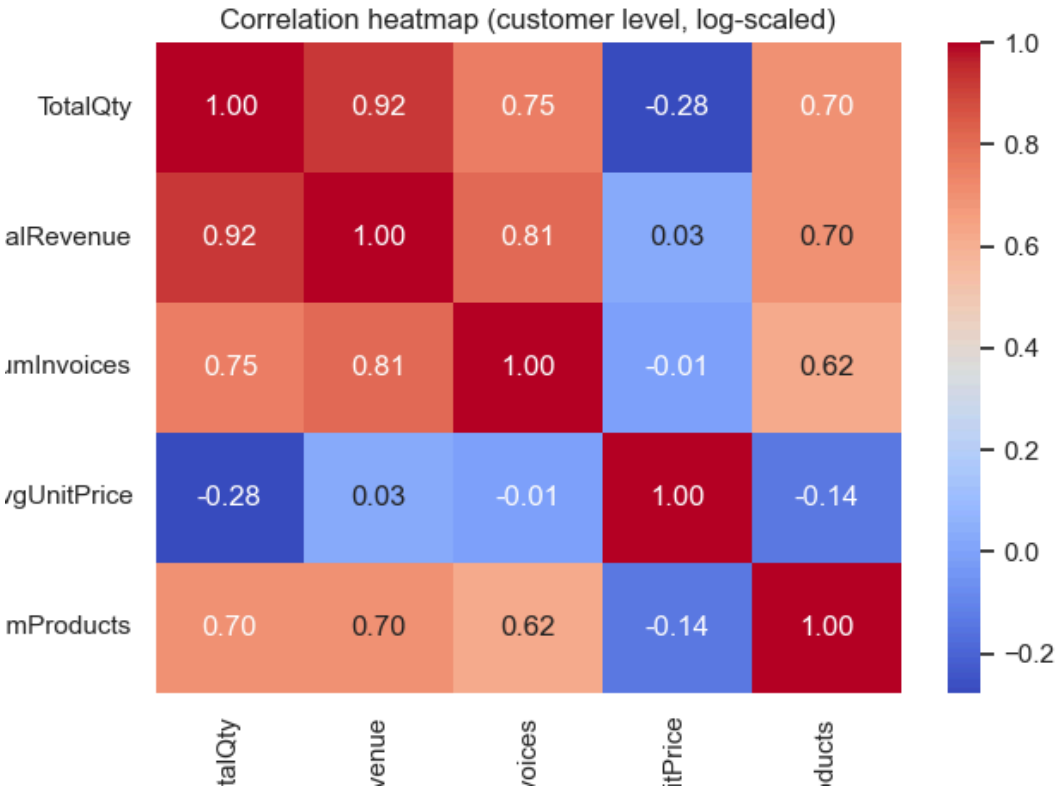

Quantity vs UnitPrice
Quantity
UnitPrice
0
20
40
60
80
100
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

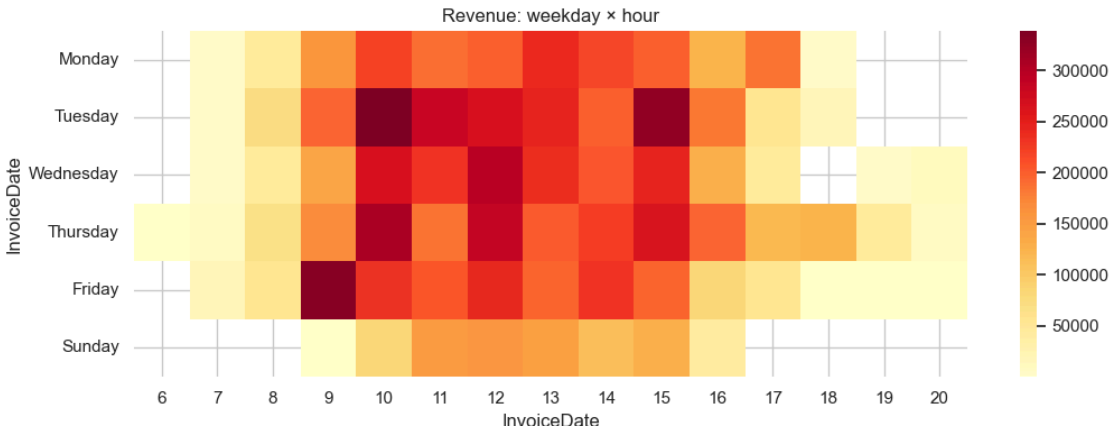

Top-10 countries by revenue
1e6
United Kingdom
Netherlands
EIRE
Germany
France
Australia
Spain
Switzerland
Belgium
Sweden
Country


Monthly revenue
1e6
Dec
Jan
2011
Feb
Mar
Apr
May
Jun
Jul
Aug
Sep
Oct
Nov
Dec
InvoiceDate


Revenue by hour of day
1e6
InvoiceDate


TotalRevenue
NumInvoices
AvgUnitPrice
NumProducts
Region
UK
Non-UK


Correlation heatmap (customer level, log-scaled)
TotalQty
1.00 0.92 0.75 -0.28 0.70
0.92 1.00 0.81 0.03 0.70
0.75 0.81 1.00 -0.01 0.62
-0.28 0.03 -0.01 1.00 -0.14
0.70 0.70 0.62 -0.14 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2



Signature of Faculty: **Grade:**

Practical –10

AIM: To perform discrete probability distribution analysis using Binomial and Poisson distributions and compare theoretical probabilities with observed data.

- **Objectives:**
  - To understand discrete probability distributions.
  - To calculate the PMF and CDF of Binomial and Poisson distributions.
  - To visualize probability distributions using graphs.
  - To compare theoretical probabilities with observed proportions or counts.

- **Theory:** Discrete probability distributions describe the probabilities of outcomes that can be counted.
  - Binomial Distribution: Models the number of successes in a fixed number of independent trials, where each trial has two possible outcomes and a constant probability of success. The Probability Mass Function (PMF) gives the probability of a particular number of successes, while the Cumulative Distribution Function (CDF) gives the probability of obtaining at most a specified number of successes.
  - Poisson Distribution: Models the number of times an event occurs within a fixed interval when events occur independently at a constant average rate. Its PMF gives the probability of a specific event count, while its CDF gives the probability of observing at most a specified count.
  - Plots of these distributions help visualize probability patterns and compare theoretical results with observed data.

- **Procedure:**
  1. Import the required statistical and visualization libraries.
  2. Load and examine the dataset.
  3. Identify a suitable binary-outcome scenario for the Binomial distribution.
  4. Calculate its PMF and CDF and plot the results.
  5. Compare theoretical probabilities with observed proportions.
  6. Identify a suitable count-based scenario for the Poisson distribution.
  7. Calculate its PMF and CDF and plot the results.
  8. Compare theoretical probabilities with observed counts and interpret the results.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, NumPy, Pandas, SciPy and Matplotlib libraries.
  - **Dataset:** Online Retail Transactions Dataset

- **Code:**

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from retail_data import load_retail

df = load_retail()
inv = (df.groupby("InvoiceNo")
         .agg(InvoiceDate=("InvoiceDate", "min"), Total=("Revenue", "sum"))
         .reset_index().sort_values("InvoiceDate"))

# ==================== (a) BINOMIAL ====================
# Experiment: pick n=10 invoices; "success" = high-value invoice (total >=
£500)
THRESH, n = 500, 10
inv["success"] = (inv.Total >= THRESH).astype(int)
p = inv.success.mean()
rng = np.random.default_rng(42)
shuffled = rng.permutation(inv.success.values)            # shuffle -> trials
~ independent
k = len(shuffled) // n
blocks = shuffled[:k * n].reshape(k, n).sum(axis=1)       # successes in each
group of 10

x = np.arange(n + 1)
pmf, cdf = stats.binom.pmf(x, n, p), stats.binom.cdf(x, n, p)
obs = np.bincount(blocks, minlength=n + 1) / k
```

```
obs_cdf = np.cumsum(obs)

print(f"p(success)={p:.4f}, n={n}, groups={k}")
print(pd.DataFrame({"x": x, "Theoretical PMF": pmf.round(4), "Observed prop":
obs.round(4),
                    "Theoretical CDF": cdf.round(4), "Observed CDF":
obs_cdf.round(4)}))
print(f"Theoretical mean={n*p:.3f}, var={n*p*(1-p):.3f} | Observed
mean={blocks.mean():.3f}, var={blocks.var():.3f}")
print(f"P(X>=3) = {1 - stats.binom.cdf(2, n, p):.4f}")

fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
w = 0.4
ax[0].bar(x - w/2, pmf, w, label="Theoretical PMF"); ax[0].bar(x + w/2, obs,
w, label="Observed proportion")
ax[0].set(title=f"Binomial PMF (n={n}, p={p:.3f})", xlabel="Successes",
ylabel="Probability"); ax[0].legend()
ax[1].step(x, cdf, where="post", label="Theoretical CDF"); ax[1].step(x,
obs_cdf, where="post", ls="--", label="Observed CDF")
ax[1].set(title="Binomial CDF", xlabel="Successes", ylabel="Cumulative
probability"); ax[1].legend()
plt.tight_layout(); plt.show()

# ==================== (b) POISSON ====================
# Experiment: number of invoices raised in each 1-hour slot (10:00-15:59) of
each trading day
inv["date"], inv["hour"] = inv.InvoiceDate.dt.date, inv.InvoiceDate.dt.hour
HOURS = range(10, 16)
cnt = inv[inv.hour.isin(HOURS)].groupby(["date", "hour"]).size()
full = pd.MultiIndex.from_product([inv.date.unique(), HOURS], names=["date",
"hour"])
cnt = cnt.reindex(full, fill_value=0)
lam = cnt.mean()

kmax = int(cnt.max())
xs = np.arange(kmax + 1)
pmf, cdf = stats.poisson.pmf(xs, lam), stats.poisson.cdf(xs, lam)
```

```
obs = np.bincount(cnt.values, minlength=kmax + 1) / len(cnt)
obs_cdf = np.cumsum(obs)

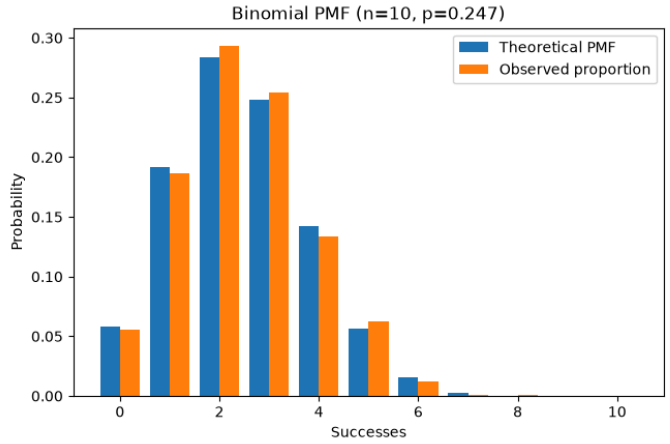
print(f"\nlambda (mean invoices/hour) = {lam:.3f}, variance = {cnt.var():.3f},
"
      f"dispersion index var/mean = {cnt.var()/lam:.2f}  (≈1 for ideal
Poisson)")
print(f"P(X=5)={stats.poisson.pmf(5, lam):.4f},  P(X<=3)={stats.poisson.cdf(3,
lam):.4f},  P(X>8)={stats.poisson.sf(8, lam):.4f}")

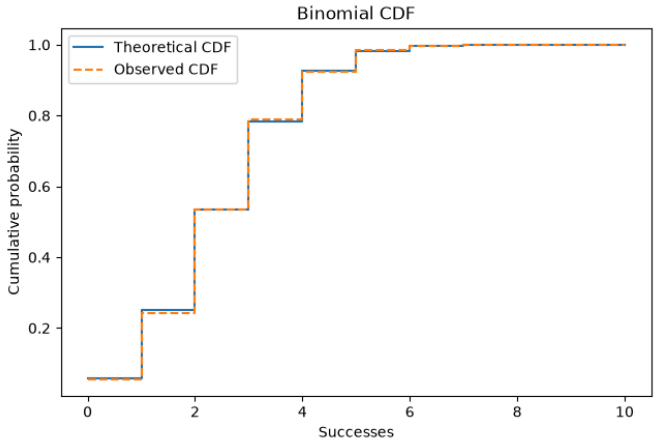
fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
ax[0].bar(xs - w/2, pmf, w, label="Theoretical PMF"); ax[0].bar(xs + w/2, obs,
w, label="Observed proportion")
ax[0].set(title=f"Poisson PMF (λ={lam:.2f})", xlabel="Invoices per hour",
ylabel="Probability"); ax[0].legend()
ax[1].step(xs, cdf, where="post", label="Theoretical CDF"); ax[1].step(xs,
obs_cdf, where="post", ls="--", label="Observed CDF")
ax[1].set(title="Poisson CDF", xlabel="Invoices per hour", ylabel="Cumulative
probability"); ax[1].legend()
plt.tight_layout(); plt.show()
print("Interpretation: if the dispersion index is well above 1, hourly counts
vary more than a pure Poisson "
      "process (arrival rate changes through the day / seasons), so the fit is
only approximate.")
```

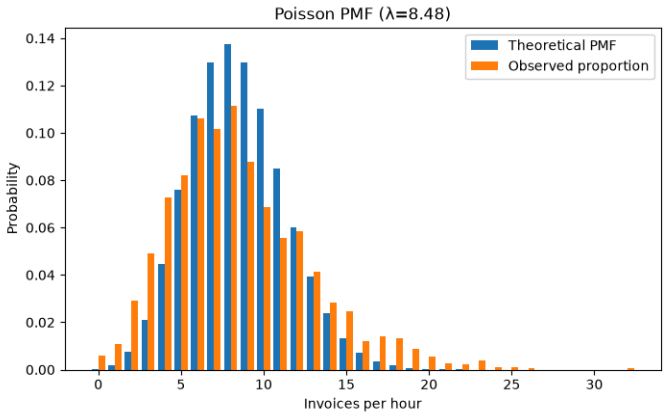
- **Output:**

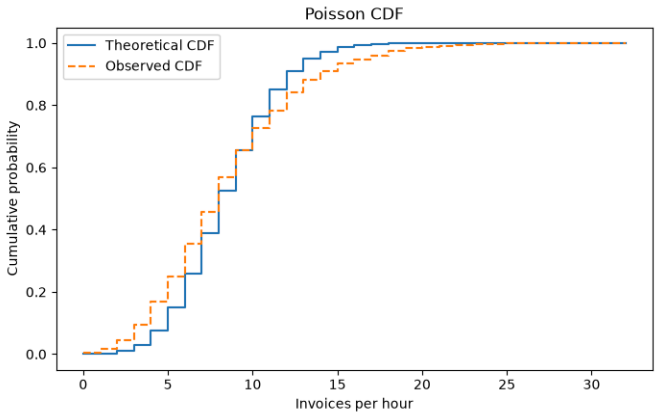
```
python .\practical_10\prac10.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_10\prac10.py
p(success)=0.2471, n=10, groups=1996
     x  Theoretical PMF  Observed prop  Theoretical CDF  Observed CDF
0    0           0.0585         0.0556           0.0585        0.0556
1    1           0.1921         0.1864           0.2506        0.2420
2    2           0.2837         0.2936           0.5343        0.5356
3    3           0.2483         0.2540           0.7826        0.7896
4    4           0.1426         0.1338           0.9252        0.9233
5    5           0.0562         0.0626           0.9814        0.9860
6    6           0.0154         0.0120           0.9967        0.9980
7    7           0.0029         0.0010           0.9996        0.9990
8    8           0.0004         0.0010           1.0000        1.0000
9    9           0.0000         0.0000           1.0000        1.0000
10  10           0.0000         0.0000           1.0000        1.0000
Theoretical mean=2.471, var=1.860 | Observed mean=2.471, var=1.793
P(X>=3) = 0.4657

lambda (mean invoices/hour) = 8.482, variance = 18.378, dispersion index
var/mean = 2.17  (≈1 for ideal Poisson)
P(X=5)=0.0758,  P(X<=3)=0.0305,  P(X>8)=0.4744
Interpretation: if the dispersion index is well above 1, hourly counts
vary more than a pure Poisson process (arrival rate changes through the
day / seasons), so the fit i
s only approximate.
```









Signature of Faculty: **Grade:**

Practical –11

AIM: To perform continuous probability distribution analysis using Normal and Exponential distributions and visualize their probability density and cumulative distribution functions.

- **Objectives:**
  - To understand continuous probability distributions.
  - To calculate the PDF and CDF of Normal and Exponential distributions.
  - To fit theoretical distribution curves over observed data.
  - To visualize and interpret the distribution of numerical attributes.

- **Theory:** Continuous probability distributions describe the probabilities of values within a continuous range. Unlike discrete distributions, probabilities are calculated over intervals rather than individual values.

  - Normal Distribution: A symmetric, bell-shaped distribution characterized by its mean and standard deviation. Its Probability Density Function (PDF) describes the distribution's density, while its Cumulative Distribution Function (CDF) gives the probability that a variable is less than or equal to a specified value.
  - Exponential Distribution: Models the waiting time between events occurring independently at a constant average rate. Its PDF represents probability density, while its CDF represents the cumulative probability up to a specified value.

  Fitting these distributions over observed data helps determine how well a theoretical model represents the actual data.

- **Procedure:**

  1. Import the required statistical and visualization libraries.
  2. Load the dataset and select suitable numerical attributes.
  3. Calculate the PDF and CDF for the Normal distribution.
  4. Plot the Normal distribution and fit its curve over the data histogram.
  5. Select a suitable time or duration attribute for the Exponential distribution.
  6. Calculate its PDF and CDF and plot the distribution.
  7. Fit the Exponential curve to the observed data.
  8. Interpret the graphs and compare the theoretical distributions with the data.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, NumPy, Pandas, SciPy and Matplotlib libraries.
  - **Dataset:** Online Retail Transactions Dataset

- **Code:**

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from retail_data import load_retail
df = load_retail()
inv = (df.groupby("InvoiceNo")
         .agg(InvoiceDate=("InvoiceDate", "min"), Total=("Revenue", "sum"))
         .reset_index().sort_values("InvoiceDate"))
# ====================== (a) NORMAL ======================
# Raw invoice totals are heavily skewed, so use log10(invoice total) which is ~normal
x = np.log10(inv.Total[inv.Total > 0])
mu, sigma = stats.norm.fit(x)
print(f"Fitted Normal on log10(Invoice Total): mu={mu:.3f}, sigma={sigma:.3f}")
print(f"Skewness={x.skew():.3f}, Excess kurtosis={x.kurt():.3f}")
print("KS test vs fitted normal:", stats.kstest(x, stats.norm(mu, sigma).cdf))
print(f"P(invoice between £50 and £500) = "
      f"{stats.norm.cdf(np.log10(500), mu, sigma) - stats.norm.cdf(np.log10(50), mu,
sigma):.4f}")
g = np.linspace(x.min(), x.max(), 400)
fig, ax = plt.subplots(1, 4, figsize=(20, 4.5))
ax[0].plot(g, stats.norm.pdf(g, mu, sigma)); ax[0].set_title("Normal PDF")
ax[1].plot(g, stats.norm.cdf(g, mu, sigma), label="Theoretical")
ax[1].plot(np.sort(x), np.arange(1, len(x) + 1) / len(x), "--", label="Empirical");
ax[1].set_title("Normal CDF"); ax[1].legend()
ax[2].hist(x, bins=50, density=True, alpha=.6, label="Data")
ax[2].plot(g, stats.norm.pdf(g, mu, sigma), "r", lw=2, label="Fitted Normal");
ax[2].set_title("Histogram + Normal fit"); ax[2].legend()
ax[2].set_xlabel("log10(Invoice Total)")
stats.probplot(x, dist="norm", plot=ax[3]); ax[3].set_title("Q-Q plot")
plt.tight_layout(); plt.show()

# ====================== (b) EXPONENTIAL ======================
# Time between consecutive invoices (minutes). Keep gaps within trading hours (0 < gap <= 60
min)
```

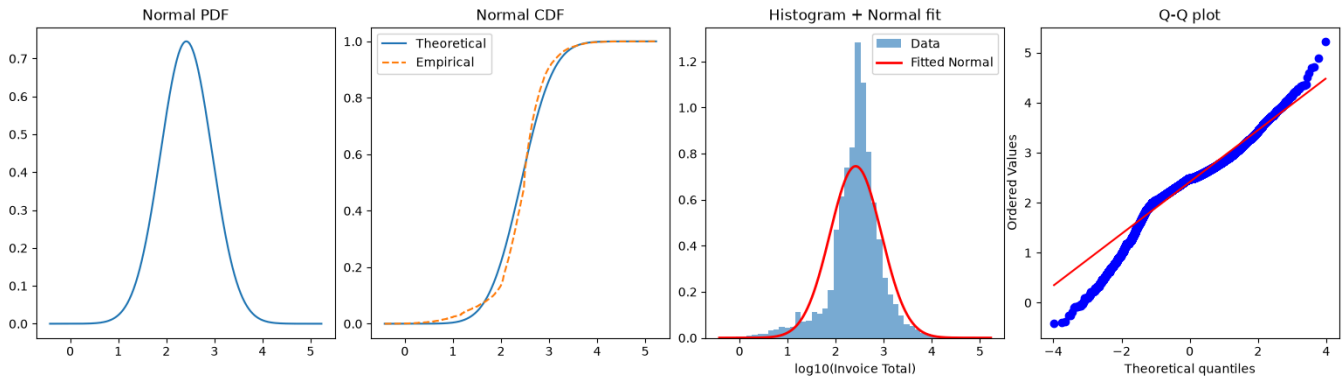
```
gaps = inv.InvoiceDate.diff().dt.total_seconds().div(60).dropna()
gaps = gaps[(gaps > 0) & (gaps <= 60)]
loc, scale = stats.expon.fit(gaps, floc=0)
lam = 1 / scale
print(f"\nFitted Exponential: scale (mean gap)={scale:.3f} min, rate λ={lam:.4f} per min")
print("KS test vs fitted exponential:", stats.kstest(gaps, "expon", args=(0, scale)))
print(f"P(next invoice arrives within 5 min)  = {stats.expon.cdf(5, 0, scale):.4f}")
print(f"P(wait more than 15 min)               = {stats.expon.sf(15, 0, scale):.4f}")

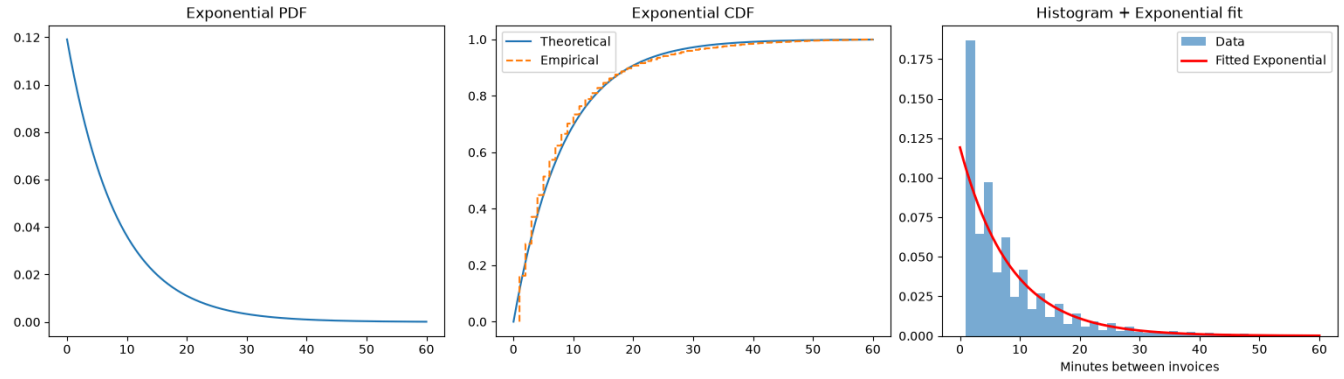
t = np.linspace(0, gaps.max(), 400)
fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
ax[0].plot(t, stats.expon.pdf(t, 0, scale)); ax[0].set_title("Exponential PDF")
ax[1].plot(t, stats.expon.cdf(t, 0, scale), label="Theoretical")
ax[1].plot(np.sort(gaps), np.arange(1, len(gaps) + 1) / len(gaps), "--", label="Empirical");
ax[1].set_title("Exponential CDF"); ax[1].legend()
ax[2].hist(gaps, bins=40, density=True, alpha=.6, label="Data")
ax[2].plot(t, stats.expon.pdf(t, 0, scale), "r", lw=2, label="Fitted Exponential")
ax[2].set(title="Histogram + Exponential fit", xlabel="Minutes between invoices");
ax[2].legend()
plt.tight_layout(); plt.show()
print("Note: timestamps are recorded to the minute, so a few ties (gap = 0) are excluded; "
      "the fit is approximate because the arrival rate varies across the day.")
```

- **Output:**

```
python .\practical_11\prac11.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_11\prac11.py
Fitted Normal on log10(Invoice Total): mu=2.415, sigma=0.535
Skewness=-0.815, Excess kurtosis=2.769
KS test vs fitted normal: KstestResult(statistic=np.float64(0.08603062401275563),
pvalue=np.float64(5.6177574299518936e-129),
statistic_location=np.float64(2.0349491466763
72), statistic_sign=np.int8(-1))
P(invoice between £50 and £500) = 0.6115

Fitted Exponential: scale (mean gap)=8.394 min, rate λ=0.1191 per min
KS test vs fitted exponential:
KstestResult(statistic=np.float64(0.1123109483221897),
pvalue=np.float64(4.543026997987513e-199), statistic_location=np.float64(1.0),
statis
tic_sign=np.int8(-1))
P(next invoice arrives within 5 min)  = 0.4488
P(wait more than 15 min)               = 0.1675
Note: timestamps are recorded to the minute, so a few ties (gap = 0) are excluded;
the fit is approximate because the arrival rate varies across the day.
```





Signature of Faculty: **Grade:**

Practical –12

AIM: To perform sampling, estimation and hypothesis testing on a given dataset using Python.

- **Objectives:**
  - To implement random, systematic and stratified sampling techniques.
  - To perform point and interval estimation.
  - To demonstrate the Central Limit Theorem using repeated sampling.
  - To conduct a one-sample t-test and interpret the results.

- **Theory:** Sampling and statistical inference help draw conclusions about a population using a smaller subset of observations.
  - **Sampling**: Random sampling selects observations randomly, systematic sampling selects observations at regular intervals, and stratified sampling divides the population into groups before selecting samples.
  - **Estimation**: Point estimation uses a single value to estimate a population parameter, while interval estimation provides a range of plausible values, often expressed as a confidence interval.
  - **Central Limit Theorem (CLT):** States that, under suitable conditions, the distribution of sample means approaches a Normal distribution as the sample size increases, even when the original population is not normally distributed.
  - **Hypothesis Testing:** A one-sample t-test determines whether the sample mean differs significantly from a specified population mean. The p-value is compared with the significance level to decide whether to reject the null hypothesis.

- **Procedure:**

1. Import the required statistical and data analysis libraries.
2. Load the dataset and identify the population of interest.
3. Apply random, systematic and stratified sampling.
4. Calculate point estimates and confidence intervals for selected attributes.
5. Draw repeated samples and plot their means to demonstrate the Central Limit Theorem.
6. Formulate the null and alternative hypotheses for a one-sample t-test.
7. Perform the t-test and compare the p-value with the significance level.
8. Interpret the results and draw conclusions.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, NumPy, SciPy and Matplotlib libraries.
  - **Dataset:** Online Retail Transactions Dataset

- **Code:**

```
import sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))

import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from retail_data import load_retail

df = load_retail()
rng = np.random.default_rng(42)
pop_mean, pop_std = df.Revenue.mean(), df.Revenue.std()
print(f"Population: N={len(df)}, mean revenue/line={pop_mean:.3f},
std={pop_std:.3f}")

# ============ (a) SAMPLING TECHNIQUES ============
n = 2000
random_s = df.sample(n=n, random_state=1)                                    #
simple random

step = len(df) // n                                                          #
systematic
start = rng.integers(0, step)
systematic_s = df.iloc[start::step]

stratified_s = df.groupby("Country").sample(frac=n / len(df), random_state=1)
# proportional stratified by Country

res = pd.DataFrame({
    "Sample size": [len(random_s), len(systematic_s), len(stratified_s)],
```

```
    "Mean Revenue": [random_s.Revenue.mean(), systematic_s.Revenue.mean(),
stratified_s.Revenue.mean()],
    "Std Revenue": [random_s.Revenue.std(), systematic_s.Revenue.std(),
stratified_s.Revenue.std()],
    "Abs error vs population mean": [abs(s.Revenue.mean() - pop_mean) for s in
(random_s, systematic_s, stratified_s)],
}, index=["Random", "Systematic", "Stratified (Country)"])
print("\n", res.round(3))
print("\nCountry share (%) - population vs stratified sample (top 5):")
print(pd.concat([df.Country.value_counts(normalize=True).mul(100).rename("Popu
lation"),

stratified_s.Country.value_counts(normalize=True).mul(100).rename("Stratified"
)],
                axis=1).head(5).round(2))

# ============ (b) POINT & INTERVAL ESTIMATION ============
samp = df.sample(n=500, random_state=7)
x = samp.Revenue
m, s, nn = x.mean(), x.std(ddof=1), len(x)
se = s / np.sqrt(nn)
print("\n--- Point estimates ---")
print(f"Mean = {m:.3f}, Std dev = {s:.3f}, Median = {x.median():.3f}")
p_hat = (samp.Country == "United Kingdom").mean()
print(f"Proportion of UK transactions (p^) = {p_hat:.3f}")

print("\n--- Interval estimates ---")
for cl in (0.90, 0.95, 0.99):
    lo, hi = stats.t.interval(cl, df=nn - 1, loc=m, scale=se)
    print(f"{int(cl*100)}% CI for mean revenue/line: ({lo:.3f}, {hi:.3f})")
z = stats.norm.ppf(0.975); me = z * np.sqrt(p_hat * (1 - p_hat) / nn)
print(f"95% CI for UK proportion: ({p_hat - me:.3f}, {p_hat + me:.3f})   [true
= {(df.Country=='United Kingdom').mean():.3f}]")
boot = [rng.choice(x, nn).mean() for _ in range(5000)]
print(f"Bootstrap 95% CI for mean: ({np.percentile(boot, 2.5):.3f},
{np.percentile(boot, 97.5):.3f})")
print(f"True population mean = {pop_mean:.3f}")
```

```
# ============ (c) CENTRAL LIMIT THEOREM ============
pop = df.Revenue.values
sizes = [2, 5, 30, 100]
fig, ax = plt.subplots(1, 5, figsize=(24, 4))
ax[0].hist(np.clip(pop, 0, np.percentile(pop, 99)), bins=60, color="grey")
ax[0].set_title("Population (skewed), clipped at 99th pct")
for a, k in zip(ax[1:], sizes):
    means = rng.choice(pop, size=(3000, k)).mean(axis=1)
    a.hist(means, bins=50, density=True, alpha=.6)
    xs = np.linspace(means.min(), means.max(), 300)
    a.plot(xs, stats.norm.pdf(xs, pop_mean, pop_std / np.sqrt(k)), "r", lw=2)
    a.set_title(f"Sample means, n={k}")
    print(f"n={k:3d}: mean of means={means.mean():.3f} (μ={pop_mean:.3f}) | "
          f"SD of means={means.std():.3f} (σ/√n={pop_std/np.sqrt(k):.3f}) |
skew={stats.skew(means):.2f}")
plt.tight_layout(); plt.show()
print("As n grows, the sampling distribution becomes bell-shaped and narrower
(σ/√n), "
      "even though the population is highly skewed -> Central Limit Theorem.")

# ============ (d) ONE-SAMPLE t-TEST ============
alpha = 0.05
test_sample = df.sample(n=500, random_state=11).Revenue
for mu0 in (20, 25):
    t, p = stats.ttest_1samp(test_sample, popmean=mu0)
    print(f"\nH0: μ = {mu0}   vs   H1: μ ≠ {mu0}")
    print(f"sample mean={test_sample.mean():.3f}, t={t:.3f}, p-value={p:.4f}")
    print("Decision:", "Reject H0 (significant difference)" if p < alpha else
"Fail to reject H0 (no significant evidence)",
          f"at α={alpha}")
t, p = stats.ttest_1samp(test_sample, 20, alternative="greater")
print(f"\nOne-sided test (H1: μ > 20): t={t:.3f}, p={p:.4f}")
```

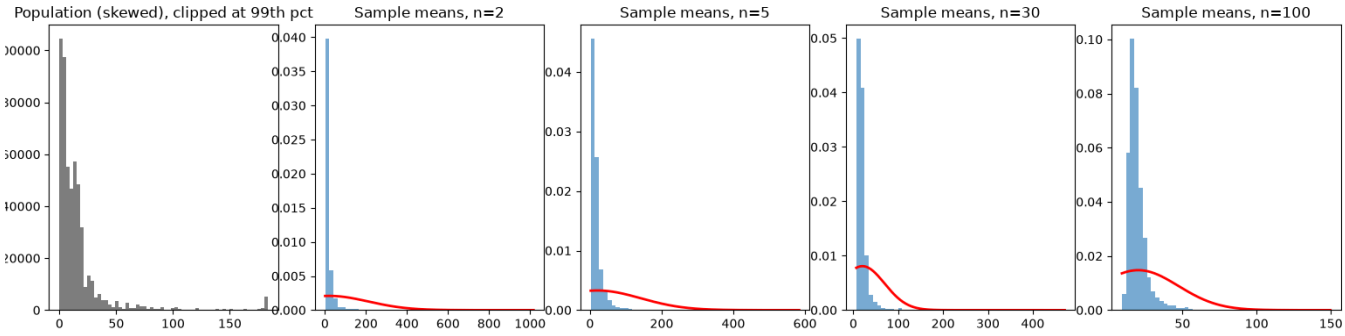
- **Output:**

```
python .\practical_12\prac12.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_12\prac12.py
Population: N=530104, mean revenue/line=20.122, std=270.357
                      Sample size  Mean Revenue  Std Revenue  Abs error vs
population mean
Random                        2000        19.546       65.780
0.575
Systematic                    2001        20.347       55.228
0.225
Stratified (Country)          2000        22.723       89.430
2.601

Country share (%) - population vs stratified sample (top 5):
                Population  Stratified
Country
United Kingdom       91.51       91.50
Germany               1.71        1.70
France                1.59        1.60
EIRE                  1.49        1.50
Spain                 0.47        0.45

--- Point estimates ---
Mean = 21.949, Std dev = 76.319, Median = 9.330
Proportion of UK transactions (pˆ) = 0.932

--- Interval estimates ---
90% CI for mean revenue/line: (16.324, 27.573)
95% CI for mean revenue/line: (15.243, 28.654)
99% CI for mean revenue/line: (13.123, 30.774)
95% CI for UK proportion: (0.910, 0.954)   [true = 0.915]
Bootstrap 95% CI for mean: (16.096, 29.602)
True population mean = 20.122
n=  2: mean of means=19.242 (μ=20.122) | SD of means=44.937 (σ/√n=191.171) |
skew=12.72
n=  5: mean of means=19.258 (μ=20.122) | SD of means=29.149 (σ/√n=120.907) |
skew=10.69
n= 30: mean of means=19.736 (μ=20.122) | SD of means=16.985 (σ/√n=49.360) |
skew=12.74
n=100: mean of means=19.591 (μ=20.122) | SD of means=7.341 (σ/√n=27.036) |
skew=4.58
As n grows, the sampling distribution becomes bell-shaped and narrower (σ/√n),
even though the population is highly skewed -> Central Limit Theorem.
H0: μ = 20   vs   H1: μ ≠ 20
sample mean=21.264, t=0.327, p-value=0.7436
Decision: Fail to reject H0 (no significant evidence) at α=0.05
H0: μ = 25   vs   H1: μ ≠ 25
sample mean=21.264, t=-0.967, p-value=0.3342
Decision: Fail to reject H0 (no significant evidence) at α=0.05
One-sided test (H1: μ > 20): t=0.327, p=0.3718
```



Signature of Faculty: **Grade:**

Practical –13

AIM: To analyze the Company Master Data of Gujarat dataset by examining its attributes, identifying missing values and computing descriptive statistics.

- **Objectives:**
  - To understand the structure and data types of the dataset.
  - To identify missing values in different attributes.
  - To implement suitable methods for handling missing data.
  - To calculate descriptive statistics for numerical attributes.

- **Theory:** Data description and statistical analysis are important steps in understanding a dataset before performing further analysis.
  - Data Types and Missing Values: Examining attribute data types helps distinguish numerical and categorical variables. Missing-value analysis identifies incomplete records, which can be handled using suitable techniques such as deletion or imputation.
  - Descriptive Statistics: Measures of central tendency include mean, median and mode. Measures of dispersion include range, variance and standard deviation. Measures of location include quartiles and percentiles, while measures of shape and symmetry include skewness and kurtosis.

  These techniques help identify the distribution, variability and quality of the dataset.

- **Procedure:**
  1. Import the required Python libraries.
  2. Load the Company Master Data of Gujarat dataset.
  3. Display the dataset's dimensions, attributes and data types.
  4. Calculate the number of missing values in each attribute.
  5. Apply suitable methods to handle missing values.
  6. Calculate descriptive statistics for numerical attributes.
  7. Interpret the statistical results and summarize the findings.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Python code editor, Pandas, NumPy and SciPy libraries.
  - **Dataset:** Company Master Data of Gujarat Datase

- **Code:**

```
import sys, os, re
import numpy as np, pandas as pd

def load_data():
    DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data",
"gujarat_companies.csv")
    df = pd.read_csv(DATA_PATH, low_memory=False, encoding_errors="replace",
                     na_values=["", " ", "NA", "N/A", "NULL", "null", "-"])
    df.columns = [re.sub(r"\W+", "_", str(c).strip()).strip("_").upper() for c
in df.columns]
    for c in df.select_dtypes("object"):
        df[c] = df[c].astype("string").str.strip()

    def find(*keys):
        for c in df.columns:
            if all(k in c for k in keys):
                return c
        raise KeyError(f"No column matching {keys}")

    C = dict(
        STATUS=find("COMPANY", "STATUS"),
        CLASS=find("COMPANY", "CLASS"),
        SUBCAT=find("SUB", "CATEGORY"),
        DATE=find("DATE", "REGISTRATION"),
        AUTH=find("AUTH"),
        PAID=find("PAID"),
        PBA=find("PRINCIPAL", "BUSINESS"),
        ROC=find("REGISTRAR"),
    )
    df[C["DATE"]] = pd.to_datetime(df[C["DATE"]], dayfirst=True,
errors="coerce")
    for k in ("AUTH", "PAID"):
        df[C[k]] = pd.to_numeric(df[C[k]], errors="coerce")
    df["YEAR"] = df[C["DATE"]].dt.year
    return df, C
```

```
pd.set_option("display.width", 200); pd.set_option("display.max_columns", 50)
df, C = load_data()
print("Detected columns:", C)

# ============ (a) DESCRIBE DATASET & DATA TYPES ============
print("Shape:", df.shape)
print(df.head())
df.info()
print("\nData type of each attribute:\n", df.dtypes)
print("\nNumeric summary:\n", df.describe().T)
print("\nCategorical summary:\n", df.select_dtypes(include=["object",
"string"]).describe().T)
print("\nUnique values per column:\n", df.nunique())
print("\nDuplicate rows:", df.duplicated().sum())

# ============ (b) MISSING VALUES ============
miss = df.isnull().sum().to_frame("Missing")
miss["Missing %"] = (miss.Missing / len(df) * 100).round(2)
print("\nMissing values:\n", miss[miss.Missing > 0].sort_values("Missing",
ascending=False))

# --- Handling methods ---
# Method 1: drop columns with > 60% missing, then drop rows without a
registration date
m1 = df.drop(columns=miss.index[miss["Missing %"] > 60])
m1 = m1.dropna(subset=[C["DATE"]])
print("\nMethod 1 (drop sparse columns / rows w/o date):", m1.shape)

# Method 2: impute - numeric -> median, categorical -> 'Unknown'
m2 = df.copy()
for c in m2.select_dtypes(include=np.number).columns:
    if c != "YEAR":
        m2[c] = m2[c].fillna(m2[c].median())
for c in m2.select_dtypes(include=["object", "string"]).columns:
    m2[c] = m2[c].fillna("Unknown")
print("Method 2 (median / 'Unknown' imputation) - missing left:",
int(m2.drop(columns=["YEAR", C["DATE"]]).isnull().sum().sum()))
```

```
# Method 3: compare imputation strategies on a numeric column
a = df[C["AUTH"]]
cmp = pd.DataFrame({"Original mean": [a.mean()],
                    "After mean-fill": [a.fillna(a.mean()).mean()],
                    "After median-fill": [a.fillna(a.median()).mean()],
                    "After ffill": [a.ffill().mean()]})
print("\nEffect of imputation on", C["AUTH"], "\n", cmp)
# Method 4: categorical -> mode
st = df[C["STATUS"]].fillna(df[C["STATUS"]].mode()[0])
# ============ (c) DESCRIPTIVE STATISTICS ============
numeric = [C["AUTH"], C["PAID"]]
rows = {}
for c in numeric:
    x = df[c].dropna()
    q1, q3 = x.quantile([.25, .75])
    rows[c] = {
        # central tendency
        "Mean": x.mean(), "Median": x.median(), "Mode": x.mode().iloc[0],
        # dispersion
        "Min": x.min(), "Max": x.max(), "Range": x.max() - x.min(),
        "Variance": x.var(), "Std Dev": x.std(), "IQR": q3 - q1,
        "Coeff. of Variation (%)": x.std() / x.mean() * 100,
        # location
        "Q1 (25th)": q1, "Q3 (75th)": q3, "P10": x.quantile(.10), "P90":
x.quantile(.90), "P99": x.quantile(.99),
        # shape & symmetry
        "Skewness": x.skew(), "Kurtosis (excess)": x.kurt(),
    }
stats_tbl = pd.DataFrame(rows)
print("\n", stats_tbl.round(2))
for c in numeric:
    sk = stats_tbl.loc["Skewness", c]
    print(f"{c}: skewness={sk:.2f} -> {'right-skewed' if sk > .5 else
'left-skewed' if sk < -.5 else 'roughly symmetric'}; "
          f"mean {'>' if stats_tbl.loc['Mean', c] > stats_tbl.loc['Median', c]
else '<='} median")
```

- **Output:**

```
python .\practical_13\prac13.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_13\prac13.py
Shape: (5060, 15)
  CORPORATE_IDENTIFICATION_NUMBER              COMPANY_NAME COMPANY_STATUS
COMPANY_CLASS           COMPANY_CATEGORY COMPANY_SUB_CATEGORY DATE_OF_REGISTRATION
REGISTERED_ST
ATE  AUTHORIZED_CAP  \
0           U58957GJ2023PTC499213      Krishna Textiles Ltd     Strike Off
Private  Company limited by Shares     Non-govt company           2023-05-25
Guja
rat        300000.0
1           U91106GJ2017PTC719877            Om Finserv Ltd         Active
Public  Company limited by Shares     Non-govt company           2017-10-02
Guja
rat      26730000.0
2           U80121GJ2017PTC290199    Jay Industries Pvt Ltd         Active
Private  Company limited by Shares     Non-govt company           2017-10-15
Guja
rat       8510000.0
3           U22696GJ2005PTC770148  Sabarmati Industries Ltd         Active
Private  Company limited by Shares     Non-govt company           2005-06-03
Guja
rat       1750000.0
4           U84652GJ1990PTC843882      Om Logistics Pvt Ltd        Dormant
Private  Company limited by Shares     Non-govt company           1990-07-12
Guja
rat        320000.0
   PAIDUP_CAPITAL             PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN
REGISTRAR_OF_COMPANIES            EMAIL_ADDR  LATEST_YEAR_ANNUAL_RETURN  YEAR
0        101000.0                   Real Estate and Renting Business
RoC-Ahmedabad  info4087@example.com                         NaN  2023
1      13861000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Ahmedabad  info1542@example.com                      2019.0  2017
2       3803000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Surat    info78@example.com                       2020.0  2017
3        433000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Ahmedabad  info2554@example.com                      2023.0  2005
4        166000.0                                            Trading
RoC-Surat  info1580@example.com                      2021.0  1990
<class 'pandas.DataFrame'>
RangeIndex: 5060 entries, 0 to 5059
Data columns (total 15 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   CORPORATE_IDENTIFICATION_NUMBER         5060 non-null   string
 1   COMPANY_NAME                            5060 non-null   string
 2   COMPANY_STATUS                          5060 non-null   string
 3   COMPANY_CLASS                           5060 non-null   string
 4   COMPANY_CATEGORY                        5060 non-null   string
```

```
 5   COMPANY_SUB_CATEGORY                      4909 non-null   string
 6   DATE_OF_REGISTRATION                      5060 non-null   datetime64[us]
 7   REGISTERED_STATE                          5060 non-null   string
 8   AUTHORIZED_CAP                            5010 non-null   float64
 9   PAIDUP_CAPITAL                            4959 non-null   float64
 10  PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  5060 non-null   string
 11  REGISTRAR_OF_COMPANIES                    5060 non-null   string
 12  EMAIL_ADDR                                4758 non-null   string
 13  LATEST_YEAR_ANNUAL_RETURN                 4305 non-null   float64
 14  YEAR                                      5060 non-null   int32
dtypes: datetime64[us](1), float64(3), int32(1), string(10)
memory usage: 573.3 KB

Data type of each attribute:
 CORPORATE_IDENTIFICATION_NUMBER                   string
COMPANY_NAME                                      string
COMPANY_STATUS                                    string
COMPANY_CLASS                                     string
COMPANY_CATEGORY                                  string
COMPANY_SUB_CATEGORY                              string
DATE_OF_REGISTRATION                        datetime64[us]
REGISTERED_STATE                                  string
AUTHORIZED_CAP                                   float64
PAIDUP_CAPITAL                                   float64
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN            string
REGISTRAR_OF_COMPANIES                            string
EMAIL_ADDR                                        string
LATEST_YEAR_ANNUAL_RETURN                        float64
YEAR                                               int32
dtype: object

Numeric summary:
                            count                          mean
min                  25%                  50%                  75%
max
            std
DATE_OF_REGISTRATION         5060  2011-08-27 04:53:58.577075  1990-01-04 00:00:00
2004-09-26 00:00:00  2013-03-07 00:00:00  2019-06-06 00:00:00  2024-12-28 00:00:00
           NaN
AUTHORIZED_CAP             5010.0             80569343.313373             100000.0
240000.0             700000.0            2070000.0        39905000000.0  149
0471778.026789
PAIDUP_CAPITAL             4959.0             47210368.421053               5000.0
102000.0             316000.0            1025000.0        30003576000.0   97
0467698.965606
LATEST_YEAR_ANNUAL_RETURN  4305.0                 2021.002323               2019.0
2020.0               2021.0               2022.0               2023.0
      1.412074
YEAR                       5060.0                 2011.156324               1990.0
2004.0               2013.0               2019.0               2024.0
      9.169521
Categorical summary:
                                       count unique
```

```
top  freq
CORPORATE_IDENTIFICATION_NUMBER         5060   5000
U62522GJ2018PTC812406     2
COMPANY_NAME                            5060    300  Shree Logistics Private
Limited    27
COMPANY_STATUS                          5060      7
Active  2984
COMPANY_CLASS                           5060      3
Private  4277
COMPANY_CATEGORY                        5060      3        Company limited by
Shares  4776
COMPANY_SUB_CATEGORY                    4909      5                  Non-govt
company  4389
REGISTERED_STATE                        5060      1
Gujarat  5060
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  5060     13                Business
Services   824
REGISTRAR_OF_COMPANIES                  5060      5
RoC-Ahmedabad  2758
EMAIL_ADDR                              4758   4700
info1185@example.com     2

Unique values per column:
 CORPORATE_IDENTIFICATION_NUMBER          5000
COMPANY_NAME                              300
COMPANY_STATUS                              7
COMPANY_CLASS                               3
COMPANY_CATEGORY                            3
COMPANY_SUB_CATEGORY                        5
DATE_OF_REGISTRATION                     3921
REGISTERED_STATE                            1
AUTHORIZED_CAP                            912
PAIDUP_CAPITAL                           1928
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN     13
REGISTRAR_OF_COMPANIES                      5
EMAIL_ADDR                               4700
LATEST_YEAR_ANNUAL_RETURN                   5
YEAR                                       35
dtype: int64

Duplicate rows: 60

Missing values:
                          Missing  Missing %
LATEST_YEAR_ANNUAL_RETURN      755      14.92
EMAIL_ADDR                     302       5.97
COMPANY_SUB_CATEGORY           151       2.98
PAIDUP_CAPITAL                 101       2.00
AUTHORIZED_CAP                  50       0.99

Method 1 (drop sparse columns / rows w/o date): (5060, 15)
Method 2 (median / 'Unknown' imputation) - missing left: 0
```

```
Effect of imputation on AUTHORIZED_CAP
    Original mean  After mean-fill  After median-fill   After ffill
0   8.056934e+07     8.056934e+07       7.978012e+07  8.443642e+07

                           AUTHORIZED_CAP  PAIDUP_CAPITAL
Mean                        8.056934e+07    4.721037e+07
Median                      7.000000e+05    3.160000e+05
Mode                        1.000000e+05    7.100000e+04
Min                         1.000000e+05    5.000000e+03
Max                         3.990500e+10    3.000358e+10
Range                       3.990490e+10    3.000357e+10
Variance                    2.221506e+18    9.418076e+17
Std Dev                     1.490472e+09    9.704677e+08
IQR                         1.830000e+06    9.230000e+05
Coeff. of Variation (%)     1.849920e+03    2.055620e+03
Q1 (25th)                   2.400000e+05    1.020000e+05
Q3 (75th)                   2.070000e+06    1.025000e+06
P10                         1.000000e+05    4.500000e+04
P90                         5.551000e+06    2.850000e+06
P99                         4.828770e+07    2.617978e+07
Skewness                    2.053000e+01    2.437000e+01
Kurtosis (excess)           4.382400e+02    6.341100e+02
AUTHORIZED_CAP: skewness=20.53 -> right-skewed; mean > median
PAIDUP_CAPITAL: skewness=24.37 -> right-skewed; mean > median
```

Signature of Faculty: **Grade:**

Practical –14

AIM: To perform filtering, grouping and statistical analysis on the Company Master Data of Gujarat dataset using specified business and registration criteria.

- **Objectives:**
  - To filter companies according to registration year and business activity.
  - To identify companies satisfying multiple conditions.
  - To group records by Company Status and Company Subcategory.
  - To calculate and compare statistics for different groups.

- **Theory:** Data filtering and grouping are essential techniques for extracting meaningful information from large datasets.
  - Filtering: Selects records that satisfy specified conditions, such as registration within a particular year range or companies belonging to a particular business activity.
  - Grouping and Aggregation: Divides records into categories based on attributes such as Company Status and Company Subcategory. Statistical operations can then be performed on each group to summarize and compare the data.

  These techniques help identify trends, compare company categories and understand the characteristics of different business groups.

- **Procedure:**

1. Import the required Python libraries.
2. Load the Company Master Data of Gujarat dataset.
3. Select companies registered between 2010 and 2015.
4. Filter finance-related companies registered after 2016.
5. Group and analyze companies according to business activity for the specified registration years.
6. Group records by Company Status and calculate relevant statistics.
7. Group records by Company Status and Company Subcategory together.
8. Display and interpret the resulting statistics.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import sys, os, re
import numpy as np, pandas as pd

def load_data():
    DATA_PATH = os.path.join(os.path.dirname(__file__), "..", "data",
"gujarat_companies.csv")
    df = pd.read_csv(DATA_PATH, low_memory=False, encoding_errors="replace",
                     na_values=["", " ", "NA", "N/A", "NULL", "null", "-"])
    df.columns = [re.sub(r"\W+", "_", str(c).strip()).strip("_").upper() for c
in df.columns]
    for c in df.select_dtypes("object"):
        df[c] = df[c].astype("string").str.strip()

    def find(*keys):
        for c in df.columns:
            if all(k in c for k in keys):
                return c
        raise KeyError(f"No column matching {keys}")

    C = dict(
        STATUS=find("COMPANY", "STATUS"),
        CLASS=find("COMPANY", "CLASS"),
        SUBCAT=find("SUB", "CATEGORY"),
        DATE=find("DATE", "REGISTRATION"),
        AUTH=find("AUTH"),
        PAID=find("PAID"),
        PBA=find("PRINCIPAL", "BUSINESS"),
        ROC=find("REGISTRAR"),
    )
    df[C["DATE"]] = pd.to_datetime(df[C["DATE"]], dayfirst=True,
```

```
errors="coerce")
    for k in ("AUTH", "PAID"):
        df[C[k]] = pd.to_numeric(df[C[k]], errors="coerce")
    df["YEAR"] = df[C["DATE"]].dt.year
    return df, C

pd.set_option("display.width", 220); pd.set_option("display.max_columns", 50);
pd.set_option("display.max_rows", 200)
df, C = load_data()
df = df.dropna(subset=[C["DATE"]])
AUTH, PAID = C["AUTH"], C["PAID"]
AGG = ["count", "mean", "median", "std", "min", "max", "sum"]

def summarize(d, title):
    print("\n" + "=" * 90 + f"\n{title}\nRecords: {len(d)}\n" + "=" * 90)
    if len(d):
        print(d[[AUTH, PAID]].agg(AGG).T.round(2))

# a) Registered between 2010 and 2015 (inclusive)
a = df[df.YEAR.between(2010, 2015)]
summarize(a, "(a) Companies registered 2010-2015")
print("\nCompanies registered per year:\n",
a.YEAR.value_counts().sort_index())

# b) Principal business = Finance AND registered after 2016 (i.e. 2017
onwards)
b = df[df[C["PBA"]].str.contains("financ", case=False, na=False) & (df.YEAR >
2016)]
summarize(b, "(b) Finance companies registered after 2016")
print("\nPrincipal-business values matched:", b[C["PBA"]].unique())

# c) By principal business activity, registered 2015-2017 (inclusive)
c = df[df.YEAR.between(2015, 2017)]
print("\n" + "=" * 90 + "\n(c) Principal business activity-wise, registered
2015-2017\n" + "=" * 90)
print(c.groupby(C["PBA"])[[AUTH, PAID]].agg(AGG).round(2))
print("\nCompany count per activity:\n", c[C["PBA"]].value_counts())
```

```
# d) Group by Company Status
print("\n" + "=" * 90 + "\n(d) Grouped by Company Status\n" + "=" * 90)
print(df.groupby(C["STATUS"])[[AUTH, PAID]].agg(AGG).round(2))
print("\nShare (%):\n",
df[C["STATUS"]].value_counts(normalize=True).mul(100).round(2))

# e) Group by Company Status + Company Sub-category
print("\n" + "=" * 90 + "\n(e) Grouped by Company Status and Sub-category\n" +
"=" * 90)
e = df.groupby([C["STATUS"], C["SUBCAT"]])[[AUTH, PAID]].agg(AGG).round(2)
print(e)
print("\nCross-tab of counts:\n", pd.crosstab(df[C["STATUS"]],
df[C["SUBCAT"]]))
```

- **Output:**

```
python .\practical_14\prac14.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_14\prac14.py
(a) Companies registered 2010-2015
Records: 1108
==========================================================================================
========
                  count          mean    median           std       min
max           sum
AUTHORIZED_CAP  1099.0  19119918.11  710000.0  5.181505e+08  100000.0
1.717500e+10  2.101279e+10
PAIDUP_CAPITAL  1087.0  13468835.33  322000.0  3.853033e+08    6000.0
1.270292e+10  1.464062e+10

Companies registered per year:
 YEAR
2010    176
2011    148
2012    190
2013    196
2014    208
2015    190
Name: count, dtype: int64

==========================================================================================
========
(b) Finance companies registered after 2016
```

```
Records: 162
================================================================================
========
                 count        mean    median          std       min           max
sum
AUTHORIZED_CAP  162.0  8773641.98  835000.0  85674430.94  100000.0  1.091540e+09
1.421330e+09
PAIDUP_CAPITAL  159.0  4619056.60  366000.0  43795241.43    8000.0  5.528010e+08
7.344300e+08

Principal-business values matched: <StringArray>
['Finance']
Length: 1, dtype: string

================================================================================
========
(c) Principal business activity-wise, registered 2015-2017
================================================================================
========
                                                    AUTHORIZED_CAP
PAIDUP_CAPITAL
                                    \
                                                             count          mean
median           std       min           max           sum          count
mean
    median           std       min
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN

Agriculture & Allied Activities                                19  2.961053e+06
600000.0  8.022119e+06  100000.0  3.533000e+07  5.626000e+07             17
1.506000e+06
  299000.0  4.489586e+06    7000.0
Business Services                                             109  1.952477e+06
910000.0  3.383602e+06  100000.0  2.400000e+07  2.128200e+08            107
1.095308e+06
  356000.0  2.379477e+06   10000.0
Community, personal & Social Services                          27  4.814815e+06
480000.0  1.432277e+07  100000.0  7.108000e+07  1.300000e+08             30
2.302500e+06
  308500.0  7.393729e+06   44000.0
Construction                                                   35  2.508000e+06
960000.0  4.736901e+06  100000.0  2.243000e+07  8.778000e+07             35
1.535314e+06
  266000.0  3.539814e+06    6000.0
Electricity, Gas & Water companies                             22  5.270455e+06
850000.0  1.549253e+07  100000.0  7.381000e+07  1.159500e+08             22
3.999364e+06
  323000.0  1.341407e+07   66000.0
Finance                                                        49  5.804286e+06
1140000.0  2.076145e+07  100000.0  1.431000e+08  2.844100e+08             49
2.511286e+06
  509000.0  7.918313e+06    5000.0
Insurance                                                       4  1.780000e+06
```

```
255000.0  3.066692e+06  230000.0  6.380000e+06  7.120000e+06             5
3.799000e+06
  192000.0  6.925084e+06  135000.0
Manufacturing (Metals & Chemicals, and products...             91  4.073846e+06
850000.0  1.217921e+07  100000.0  1.099400e+08  3.707200e+08             90
2.312667e+06
  490500.0  6.841335e+06    8000.0
Manufacturing (Textiles)                                       46  2.164565e+06
890000.0  3.324751e+06  100000.0  1.885000e+07  9.957000e+07             45
7.984444e+05
  519000.0  9.048222e+05   34000.0
Mining & Quarrying                                             12  3.063333e+06
1950000.0  3.456123e+06  100000.0  1.067000e+07  3.676000e+07             12
1.197667e+06
  610000.0  1.519663e+06   65000.0
Real Estate and Renting Business                               48  1.349844e+09
620000.0  6.534280e+09  100000.0  3.235900e+10  6.479250e+10             47
1.277429e+09
  291000.0  6.121476e+09    9000.0
Trading                                                        81  1.991605e+06
520000.0  4.013152e+06  100000.0  1.970000e+07  1.613200e+08             82
1.024537e+06
  255000.0  2.543780e+06   10000.0
Transport, storage and Communications                          29  1.493103e+06
550000.0  2.398363e+06  100000.0  1.139000e+07  4.330000e+07             28
5.485714e+05
  232000.0  7.424095e+05    7000.0

                                                              max           sum
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN
Agriculture & Allied Activities                      1.885200e+07  2.560200e+07
Business Services                                    1.635800e+07  1.171980e+08
Community, personal & Social Services                4.012200e+07  6.907500e+07
Construction                                         1.807900e+07  5.373600e+07
Electricity, Gas & Water companies                   6.356400e+07  8.798600e+07
Finance                                              5.257600e+07  1.230530e+08
Insurance                                            1.605700e+07  1.899500e+07
Manufacturing (Metals & Chemicals, and products...   5.961600e+07  2.081400e+08
Manufacturing (Textiles)                             4.297000e+06  3.593000e+07
Mining & Quarrying                                   4.585000e+06  1.437200e+07
Real Estate and Renting Business                     3.000358e+10  6.003918e+10
Trading                                              1.799100e+07  8.401200e+07
Transport, storage and Communications                2.550000e+06  1.536000e+07

Company count per activity:
 PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN
Business Services                                          110
Manufacturing (Metals & Chemicals, and products thereof)    93
Trading                                                     83
Finance                                                     49
Real Estate and Renting Business                            49
Manufacturing (Textiles)                                    46
Construction                                                35
```

```
Community, personal & Social Services                          30
Transport, storage and Communications                          29
Electricity, Gas & Water companies                             22
Agriculture & Allied Activities                                19
Mining & Quarrying                                             12
Insurance                                                       5
Name: count, dtype: Int64

================================================================================
========
(d) Grouped by Company Status
================================================================================
========
                              AUTHORIZED_CAP
PAIDUP_CAPITAL

                                       count          mean    median           std
min           max           sum          count          mean    median           s
td     min           max           sum
COMPANY_STATUS

Active                                  2952  8.265288e+07  710000.0  1.511620e+09
100000.0  3.407500e+10  2.439913e+11          2923  5.175391e+07  317000.0
1.086362e+
09  5000.0  3.000358e+10  1.512767e+11
Amalgamated                              148  5.698108e+06  800000.0  2.845841e+07
100000.0  2.733800e+08  8.433200e+08           146  1.978027e+07  397500.0
2.064303e+
08  7000.0  2.491272e+09  2.887919e+09
Converted to LLP                         200  2.530378e+08  680000.0  2.520952e+09
100000.0  2.895600e+10  5.060757e+10           203  2.077935e+08  291000.0
2.081231e+
09  9000.0  2.276780e+10  4.218208e+10
Dormant                                  255  2.686431e+06  670000.0  7.659314e+06
100000.0  8.078000e+07  6.850400e+08           251  1.378693e+06  300000.0
4.225439e+
06  5000.0  5.429200e+07  3.460520e+08
Strike Off                               973  1.089372e+08  660000.0  1.788053e+09
100000.0  3.990500e+10  1.059959e+11           963  3.801106e+07  295000.0
5.881899e+
08  6000.0  1.270292e+10  3.660465e+10
Under Liquidation                        194  4.412835e+06  975000.0  1.675371e+07
100000.0  1.715200e+08  8.560900e+08           189  2.431180e+06  455000.0
1.059254e+
07  8000.0  1.295560e+08  4.594930e+08
Under Process of Striking Off            288  2.337361e+06  655000.0  5.826835e+06
100000.0  7.149000e+07  6.731600e+08           284  1.265331e+06  294000.0
3.736225e+
06  7000.0  4.901000e+07  3.593540e+08
Share (%):
 COMPANY_STATUS
Active                          58.97
Strike Off                      19.39
```

```
Under Process of Striking Off     5.73
Dormant                           5.04
Converted to LLP                  4.07
Under Liquidation                 3.85
Amalgamated                       2.94
Name: proportion, dtype: Float64

================================================================================
========
(e) Grouped by Company Status and Sub-category
================================================================================
========
                                                                AUTHORIZED_CAP
PAIDUP_CAPITAL
                                                          \
count          mean     median           std        min           max
sum          count
       mean     median           std       min
COMPANY_STATUS                COMPANY_SUB_CATEGORY

Active                        Guarantee and Association comp             85
3.023176e+06   650000.0  9.765173e+06   100000.0  8.631000e+07  2.569700e+08
85  1
.393388e+06   229000.0  3.929512e+06    6000.0
Non-govt company                         2553  7.567587e+07   710000.0
1.504923e+09   100000.0  3.407500e+10  1.932005e+11           2527  5
.294358e+07   316000.0  1.140586e+09    5000.0
State govt company                         67  1.739858e+08  1260000.0
1.397849e+09   100000.0  1.144500e+10  1.165705e+10             67  1
.557627e+06   348000.0  3.684704e+06    9000.0
Subsidiary of Foreign Company            122  3.161991e+08   560000.0
2.612120e+09   100000.0  2.647400e+10  3.857629e+10            121  1
.416447e+08   371000.0  1.157010e+09   10000.0
Union Govt company                        35  1.333714e+06   790000.0
2.229781e+06   100000.0  1.295000e+07  4.668000e+07             33  6
.771212e+05   342000.0  1.350068e+06    8000.0
Amalgamated                   Guarantee and Association comp              5
1.114000e+06   940000.0  1.016356e+06   180000.0  2.810000e+06  5.570000e+06
5  7
.648000e+05   433000.0  8.163974e+05   81000.0
Non-govt company                         133  6.251203e+06   850000.0
2.998033e+07   100000.0  2.733800e+08  8.314100e+08            131  2
.199282e+07   406000.0  2.179045e+08    7000.0
State govt company                         2  9.150000e+05   915000.0
1.152584e+06   100000.0  1.730000e+06  1.830000e+06             2  1
.350000e+05   135000.0  9.333810e+04   69000.0
Subsidiary of Foreign Company              3  3.733333e+05   340000.0
1.724336e+05   220000.0  5.600000e+05  1.120000e+06             3  1
.956667e+05   221000.0  6.388531e+04  123000.0
Union Govt company                         2  5.650000e+05   565000.0
1.060660e+05   490000.0  6.400000e+05  1.130000e+06             2  3
.320000e+05   332000.0  9.050967e+04  268000.0
Converted to LLP              Guarantee and Association comp              5
```

```
5.628000e+06   600000.0  1.133184e+07   100000.0  2.589000e+07  2.814000e+07
5  3
.490000e+06   405000.0  7.073567e+06   41000.0
Non-govt company                            181  2.792953e+08   690000.0
2.649287e+09   100000.0  2.895600e+10  5.055245e+10             185  2
.278388e+08   291000.0  2.179611e+09    9000.0
State govt company                            2  3.100000e+05   310000.0
9.899495e+04   240000.0  3.800000e+05  6.200000e+05               2  1
.205000e+05   120500.0  1.251579e+05   32000.0
Subsidiary of Foreign Company                 6  1.388333e+06  1055000.0
1.265566e+06   390000.0  3.700000e+06  8.330000e+06               5  4
.806000e+05   344000.0  4.477754e+05  133000.0
Union Govt company                            1  5.700000e+05   570000.0
NaN   570000.0  5.700000e+05  5.700000e+05              1  1
.620000e+05   162000.0           NaN  162000.0
Dormant                       Guarantee and Association comp               8
4.683750e+06  1960000.0  7.479922e+06   100000.0  2.264000e+07  3.747000e+07
8  1
.109500e+06   579000.0  1.262056e+06   10000.0
Non-govt company                            226  2.751549e+06   660000.0
7.993826e+06   100000.0  8.078000e+07  6.218500e+08             222  1
.463869e+06   299500.0  4.475263e+06    5000.0
State govt company                            3  1.040000e+06   900000.0
8.388087e+05   280000.0  1.940000e+06  3.120000e+06               3  7
.013333e+05   780000.0  4.867908e+05  180000.0
Subsidiary of Foreign Company                 9  4.644444e+05   470000.0
2.404221e+05   100000.0  8.200000e+05  4.180000e+06               9  2
.505556e+05   251000.0  1.247088e+05   38000.0
Union Govt company                            3  8.866667e+05   420000.0
9.865766e+05   220000.0  2.020000e+06  2.660000e+06               3  3
.710000e+05   287000.0  3.711981e+05   49000.0
Strike Off                    Guarantee and Association comp              33
3.166364e+06   610000.0  5.730492e+06   100000.0  2.708000e+07  1.044900e+08
33  2
.028758e+06   383000.0  4.776489e+06   12000.0
Non-govt company                            844  1.249989e+08   670000.0
1.919480e+09   100000.0  3.990500e+10  1.054991e+11             834  4
.353749e+07   293500.0  6.319055e+08    6000.0
State govt company                           19  2.109474e+06   750000.0
2.788429e+06   100000.0  9.230000e+06  4.008000e+07              18  9
.018333e+05   383000.0  1.358728e+06   26000.0
Subsidiary of Foreign Company                41  2.799268e+06   430000.0
1.145052e+07   100000.0  7.381000e+07  1.147700e+08              41  2
.023073e+06   166000.0  9.882937e+06   17000.0
Union Govt company                           10  1.024000e+06   465000.0
1.059593e+06   140000.0  3.180000e+06  1.024000e+07              11  4
.305455e+05   286000.0  5.179900e+05   30000.0
Under Liquidation             Guarantee and Association comp               4
3.832500e+07  4370000.0  6.987726e+07  1460000.0  1.431000e+08  1.533000e+08
4  1
.424050e+07  1729000.0  2.556205e+07  928000.0
Non-govt company                            171  3.915965e+06   970000.0
1.426842e+07   100000.0  1.715200e+08  6.696300e+08             165  2
```

```
.287958e+06   455000.0  1.060743e+07    8000.0
State govt company                          6  8.566667e+05   240000.0
1.028215e+06   100000.0  2.270000e+06  5.140000e+06             6  5
.586667e+05   182500.0  6.783228e+05   61000.0
Subsidiary of Foreign Company              11  2.339091e+06   750000.0
3.460851e+06   130000.0  1.180000e+07  2.573000e+07            12  1
.780333e+06   577000.0  2.709711e+06   21000.0
Under Process of Striking Off Guarantee and Association comp             9
2.404444e+06   970000.0  3.121454e+06   100000.0  9.350000e+06  2.164000e+07
8  6
.306250e+05   261000.0  8.336136e+05   11000.0
Non-govt company                          236  2.556525e+06   660000.0
6.362413e+06   100000.0  7.149000e+07  6.033400e+08            233  1
.439318e+06   339000.0  4.097423e+06    7000.0
State govt company                         11  1.581818e+06   700000.0
1.636012e+06   120000.0  4.820000e+06  1.740000e+07             11  5
.479091e+05   329000.0  4.887121e+05   76000.0
Subsidiary of Foreign Company               9  9.144444e+05   810000.0
8.934639e+05   100000.0  2.940000e+06  8.230000e+06              9  4
.374444e+05   314000.0  4.293929e+05   34000.0
Union Govt company                          4  1.200000e+05   110000.0
2.708013e+04   100000.0  1.600000e+05  4.800000e+05              4  6
.525000e+04    63000.0  1.203813e+04   54000.0

                                                                      max
sum
COMPANY_STATUS                COMPANY_SUB_CATEGORY
Active                        Guarantee and Association comp  3.421200e+07
1.184380e+08
                              Non-govt company                3.000358e+10
1.337884e+11
                              State govt company              2.630000e+07
1.043610e+08
                              Subsidiary of Foreign Company   1.161036e+10
1.713901e+10
                              Union Govt company              7.796000e+06
2.234500e+07
Amalgamated                   Guarantee and Association comp  2.144000e+06
3.824000e+06
                              Non-govt company                2.491272e+09
2.881060e+09
                              State govt company              2.010000e+05
2.700000e+05
                              Subsidiary of Foreign Company   2.430000e+05
5.870000e+05
                              Union Govt company              3.960000e+05
6.640000e+05
Converted to LLP              Guarantee and Association comp  1.614000e+07
1.745000e+07
                              Non-govt company                2.276780e+10
4.215018e+10
                              State govt company              2.090000e+05
2.410000e+05
```

```
                              Subsidiary of Foreign Company   1.264000e+06
2.403000e+06
                              Union Govt company              1.620000e+05
1.620000e+05
Dormant                       Guarantee and Association comp  3.626000e+06
8.876000e+06
                              Non-govt company                5.429200e+07
3.249790e+08
                              State govt company              1.144000e+06
2.104000e+06
                              Subsidiary of Foreign Company   4.570000e+05
2.255000e+06
                              Union Govt company              7.770000e+05
1.113000e+06
Strike Off                    Guarantee and Association comp  2.498900e+07
6.694900e+07
                              Non-govt company                1.270292e+10
3.631027e+10
                              State govt company              4.212000e+06
1.623300e+07
                              Subsidiary of Foreign Company   6.356400e+07
8.294600e+07
                              Union Govt company              1.905000e+06
4.736000e+06
Under Liquidation             Guarantee and Association comp  5.257600e+07
5.696200e+07
                              Non-govt company                1.295560e+08
3.775130e+08
                              State govt company              1.705000e+06
3.352000e+06
                              Subsidiary of Foreign Company   9.451000e+06
2.136400e+07
Under Process of Striking Off Guarantee and Association comp  2.262000e+06
5.045000e+06
                              Non-govt company                4.901000e+07
3.353610e+08
                              State govt company              1.649000e+06
6.027000e+06
                              Subsidiary of Foreign Company   1.145000e+06
3.937000e+06
                              Union Govt company              8.100000e+04
2.610000e+05

Cross-tab of counts:
 COMPANY_SUB_CATEGORY           Guarantee and Association comp  Non-govt company
State govt company  Subsidiary of Foreign Company  Union Govt company
COMPANY_STATUS
Active                                                     85              2582
68                             124                  35
Amalgamated                                                 5               134
2                               3                   2
Converted to LLP                                            5               187
2                               6                   1
```

```
Dormant                                                 8                226
3                          9                    3
Strike Off                                             33                851
19                         41                   11
Under Liquidation                                       4                171
6                         12                    0
Under Process of Striking Off                           9                238
11                         9                    4
```

Signature of Faculty: **Grade:**

Practical –15

AIM: To perform data preparation operations on the Company Master Data of Gujarat dataset using Python.

- **Objectives:**
  - To load and inspect data.
  - To identify and remove duplicate records.
  - To perform slicing, dicing, filtering and selection.
  - To concatenate and transform data.
  - To add, remove, sort and shuffle records and variables.

- **Theory:** Data preparation involves organizing and modifying raw data into a suitable structure for analysis. It includes removing duplicate records, selecting relevant rows and columns, filtering records using conditions, and transforming data into appropriate formats. Concatenation combines multiple datasets or DataFrames, while adding or removing cases and variables changes the structure of the data. Sorting arranges records according to specified attributes, whereas shuffling changes their order. These operations improve data organization and prepare it for further analysis.

- **Procedure:**

  1. Import the required Python libraries.
  2. Load the Company Master Data of Gujarat dataset into a DataFrame.
  3. Inspect the dataset and remove duplicate records.
  4. Perform slicing, dicing, filtering and selection operations.
  5. Concatenate suitable datasets or DataFrames.
  6. Transform selected attributes as required.
  7. Add or remove records and variables.
  8. Sort and shuffle the records.
  9. Display the prepared dataset and verify the changes.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
from pathlib import Path
import pandas as pd

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)

# ------------------------------------------------------------------ 1. LOADING
# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
print("=== 1. DATA LOADING ===")
print("Shape :", df.shape)
print(df.head(), "\n")
df.info()

# ------------------------------------------------------------ 2. REMOVE DUPLICATES
print("\n=== 2. REMOVAL OF DUPLICATES ===")
print("Duplicate rows before :", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)
print("Duplicate rows after  :", df.duplicated().sum())
print("Shape after removal   :", df.shape)

# ------------------------------------------------------------ 3. SLICING AND DICING
print("\n=== 3. SLICING AND DICING ===")
print("First 5 rows, first 4 columns (iloc):")
print(df.iloc[:5, :4])
print("\nRows 10-14, selected columns (loc):")
print(df.loc[10:14, ["COMPANY_NAME", "COMPANY_STATUS", "AUTHORIZED_CAP"]])
print("\nEvery 1000th record (step slicing):")
print(df.iloc[::1000, [1, 2, 3]])

# ------------------------------------------------------ 4. FILTERING AND SELECTING
print("\n=== 4. FILTERING AND SELECTING ===")
active = df[df["COMPANY_STATUS"] == "Active"]
```

```
print("Active companies:", len(active))
big_public = df[(df["COMPANY_CLASS"] == "Public") & (df["AUTHORIZED_CAP"] >
10_000_000)]
print("Public companies with authorized capital > 1 crore:", len(big_public))
selected = df[df["PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN"].isin(["Finance",
"Insurance"])]
print("Finance or Insurance companies:", len(selected))
print("Companies with 'Textiles' in name:",
      df["COMPANY_NAME"].str.contains("Textiles", case=False).sum())
print("\nSelected columns only:")
print(df[["COMPANY_NAME", "COMPANY_CLASS", "AUTHORIZED_CAP"]].head())
# ------------------------------------------- 5. CONCATENATING AND
TRANSFORMING
print("\n=== 5. CONCATENATING AND TRANSFORMING ===")
# (a) concatenate rows: stack two subsets back together
private_df = df[df["COMPANY_CLASS"] == "Private"].head(3)
public_df = df[df["COMPANY_CLASS"] == "Public"].head(3)
stacked = pd.concat([private_df, public_df], axis=0, ignore_index=True)
print("Row-wise concat (private + public):")
print(stacked[["COMPANY_NAME", "COMPANY_CLASS"]])
# (b) concatenate columns: join a lookup table on a key
roc_info = pd.DataFrame({
    "REGISTRAR_OF_COMPANIES": ["RoC-Ahmedabad", "RoC-Gandhinagar",
"RoC-Vadodara",
                               "RoC-Surat", "RoC-Rajkot"],
    "ZONE": ["Central", "Capital", "East", "South", "Saurashtra"]})
df = df.merge(roc_info, on="REGISTRAR_OF_COMPANIES", how="left")
print("\nColumn-wise merge (added ZONE):")
print(df[["REGISTRAR_OF_COMPANIES", "ZONE"]].drop_duplicates())
# (c) transformations
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
df["COMPANY_NAME"] = df["COMPANY_NAME"].str.strip().str.title()
df["AUTH_CAP_LAKHS"] = df["AUTHORIZED_CAP"] / 1e5
df["COMPANY_STATUS"] = df["COMPANY_STATUS"].astype("category")
```

```
print("\nTransformed columns (REG_YEAR, AUTH_CAP_LAKHS):")
print(df[["DATE_OF_REGISTRATION", "REG_YEAR", "AUTHORIZED_CAP",
"AUTH_CAP_LAKHS"]].head())
# ------------------------------------ 6. ADDING AND REMOVING CASES/VARIABLES
print("\n=== 6. ADDING AND REMOVING CASES / VARIABLES ===")
# add variable
df["COMPANY_AGE"] = 2026 - df["REG_YEAR"]
df["CAPITAL_SIZE"] = pd.cut(df["AUTHORIZED_CAP"],
                            bins=[0, 1e6, 1e7, 1e8, float("inf")],
                            labels=["Small", "Medium", "Large", "Very Large"])
print("Added COMPANY_AGE and CAPITAL_SIZE:")
print(df[["COMPANY_NAME", "COMPANY_AGE", "CAPITAL_SIZE"]].head())
# remove variables
before = df.shape[1]
df = df.drop(columns=["REGISTERED_STATE", "EMAIL_ADDR"])
print(f"\nColumns before: {before}, after dropping 2 columns: {df.shape[1]}")
# add a case (row)
new_row = {c: None for c in df.columns}
new_row.update({"COMPANY_NAME": "Demo New Company Pvt Ltd", "COMPANY_STATUS":
"Active",
                "COMPANY_CLASS": "Private", "AUTHORIZED_CAP": 500000})
rows_before = len(df)
df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
print(f"Rows before adding case: {rows_before}, after: {len(df)}")
# remove cases (rows)
df = df[df["COMPANY_NAME"] != "Demo New Company Pvt Ltd"]
df = df.dropna(subset=["DATE_OF_REGISTRATION"]).reset_index(drop=True)
print("Rows after removing demo row / rows with no date:", len(df))
# ------------------------------------------------ 7. SORTING AND SHUFFLING
print("\n=== 7. SORTING AND SHUFFLING ===")
print("Top 5 by Authorized Capital (descending):")
print(df.sort_values("AUTHORIZED_CAP", ascending=False)[
    ["COMPANY_NAME", "AUTHORIZED_CAP"]].head())
print("\nSorted by Class (asc) then Capital (desc):")
print(df.sort_values(["COMPANY_CLASS", "AUTHORIZED_CAP"], ascending=[True,
False])[
    ["COMPANY_NAME", "COMPANY_CLASS", "AUTHORIZED_CAP"]].head())
```

```
shuffled = df.sample(frac=1, random_state=42).reset_index(drop=True)
print("\nAfter shuffling (first 5 names):")
print(shuffled["COMPANY_NAME"].head().tolist())
# ---------------------------------------------------------------- SAVE
df.to_csv("gujarat_companies_prepared.csv", index=False)
print("\nPrepared data saved to gujarat_companies_prepared.csv  Shape:",
df.shape)
```

- **Output:**

```
python .\practical_15\practical_15.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_15\practical_15.py
=== 1. DATA LOADING ===
Shape : (5060, 14)
  CORPORATE_IDENTIFICATION_NUMBER              COMPANY_NAME COMPANY_STATUS
COMPANY_CLASS          COMPANY_CATEGORY COMPANY_SUB_CATEGORY DATE_OF_REGISTRATION
REGISTERED_ST
ATE  AUTHORIZED_CAP  \
0           U58957GJ2023PTC499213      Krishna Textiles Ltd     Strike Off
Private  Company limited by Shares     Non-govt company           25-05-2023
Guja
rat        300000.0
1           U91106GJ2017PTC719877            Om Finserv Ltd         Active
Public  Company limited by Shares     Non-govt company           02-10-2017
Guja
rat      26730000.0
2           U80121GJ2017PTC290199    Jay Industries Pvt Ltd         Active
Private  Company limited by Shares     Non-govt company           15-10-2017
Guja
rat       8510000.0
3           U22696GJ2005PTC770148  Sabarmati Industries Ltd         Active
Private  Company limited by Shares     Non-govt company           03-06-2005
Guja
rat       1750000.0
4           U84652GJ1990PTC843882      Om Logistics Pvt Ltd        Dormant
Private  Company limited by Shares     Non-govt company           12-07-1990
Guja
rat        320000.0
   PAIDUP_CAPITAL             PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN
REGISTRAR_OF_COMPANIES            EMAIL_ADDR  LATEST_YEAR_ANNUAL_RETURN
0        101000.0                  Real Estate and Renting Business
RoC-Ahmedabad  info4087@example.com                         NaN
1      13861000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Ahmedabad  info1542@example.com                      2019.0
2       3803000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Surat    info78@example.com                    2020.0
```

```
3        433000.0  Manufacturing (Metals & Chemicals, and product...
RoC-Ahmedabad  info2554@example.com                      2023.0
4        166000.0                                             Trading
RoC-Surat  info1580@example.com                     2021.0

<class 'pandas.DataFrame'>
RangeIndex: 5060 entries, 0 to 5059
Data columns (total 14 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   CORPORATE_IDENTIFICATION_NUMBER         5060 non-null   str
 1   COMPANY_NAME                            5060 non-null   str
 2   COMPANY_STATUS                          5060 non-null   str
 3   COMPANY_CLASS                           5060 non-null   str
 4   COMPANY_CATEGORY                        5060 non-null   str
 5   COMPANY_SUB_CATEGORY                    4909 non-null   str
 6   DATE_OF_REGISTRATION                    5060 non-null   str
 7   REGISTERED_STATE                        5060 non-null   str
 8   AUTHORIZED_CAP                          5010 non-null   float64
 9   PAIDUP_CAPITAL                          4959 non-null   float64
 10  PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  5060 non-null   str
 11  REGISTRAR_OF_COMPANIES                  5060 non-null   str
 12  EMAIL_ADDR                              4758 non-null   str
 13  LATEST_YEAR_ANNUAL_RETURN               4305 non-null   float64
dtypes: float64(3), str(11)
memory usage: 553.6 KB

=== 2. REMOVAL OF DUPLICATES ===
Duplicate rows before : 60
Duplicate rows after  : 0
Shape after removal   : (5000, 14)

=== 3. SLICING AND DICING ===
First 5 rows, first 4 columns (iloc):
  CORPORATE_IDENTIFICATION_NUMBER              COMPANY_NAME COMPANY_STATUS
COMPANY_CLASS
0          U58957GJ2023PTC499213      Krishna Textiles Ltd     Strike Off
Private
1          U91106GJ2017PTC719877             Om Finserv Ltd         Active
Public
2          U80121GJ2017PTC290199    Jay Industries Pvt Ltd         Active
Private
3          U22696GJ2005PTC770148  Sabarmati Industries Ltd         Active
Private
4          U84652GJ1990PTC843882      Om Logistics Pvt Ltd        Dormant
Private

Rows 10-14, selected columns (loc):
                    COMPANY_NAME COMPANY_STATUS  AUTHORIZED_CAP
10  Aaryan Realty Private Limited     Strike Off        100000.0
11           Shree Industries Ltd         Active        150000.0
12     Sabarmati Textiles Pvt Ltd    Amalgamated       3210000.0
13         Aaryan Traders Pvt Ltd     Strike Off        780000.0
```

```
14            Shree Infra Pvt Ltd         Active       3730000.0

Every 1000th record (step slicing):
                          COMPANY_NAME     COMPANY_STATUS COMPANY_CLASS
0                Krishna Textiles Ltd          Strike Off       Private
1000            Aaryan Traders Pvt Ltd             Active       Private
2000  Krishna Textiles Private Limited  Under Liquidation       Private
3000           Sardar Textiles Pvt Ltd             Active       Private
4000           Narmada Enterprises Ltd             Active       Private

=== 4. FILTERING AND SELECTING ===
Active companies: 2941
Public companies with authorized capital > 1 crore: 148
Finance or Insurance companies: 473
Companies with 'Textiles' in name: 486

Selected columns only:
               COMPANY_NAME COMPANY_CLASS  AUTHORIZED_CAP
0      Krishna Textiles Ltd       Private        300000.0
1             Om Finserv Ltd        Public      26730000.0
2    Jay Industries Pvt Ltd       Private       8510000.0
3  Sabarmati Industries Ltd       Private       1750000.0
4      Om Logistics Pvt Ltd       Private        320000.0

=== 5. CONCATENATING AND TRANSFORMING ===
Row-wise concat (private + public):
                 COMPANY_NAME COMPANY_CLASS
0        Krishna Textiles Ltd       Private
1      Jay Industries Pvt Ltd       Private
2    Sabarmati Industries Ltd       Private
3              Om Finserv Ltd        Public
4            Sardar Infra Ltd        Public
5  Sabarmati Textiles Pvt Ltd        Public

Column-wise merge (added ZONE):
   REGISTRAR_OF_COMPANIES        ZONE
0           RoC-Ahmedabad     Central
2               RoC-Surat       South
12            RoC-Vadodara        East
16        RoC-Gandhinagar     Capital
27              RoC-Rajkot  Saurashtra

Transformed columns (REG_YEAR, AUTH_CAP_LAKHS):
  DATE_OF_REGISTRATION  REG_YEAR  AUTHORIZED_CAP  AUTH_CAP_LAKHS
0           2023-05-25      2023        300000.0             3.0
1           2017-10-02      2017      26730000.0           267.3
2           2017-10-15      2017       8510000.0            85.1
3           2005-06-03      2005       1750000.0            17.5
4           1990-07-12      1990        320000.0             3.2

=== 6. ADDING AND REMOVING CASES / VARIABLES ===
Added COMPANY_AGE and CAPITAL_SIZE:
               COMPANY_NAME  COMPANY_AGE CAPITAL_SIZE
```

```
0      Krishna Textiles Ltd            3        Small
1            Om Finserv Ltd            9        Large
2    Jay Industries Pvt Ltd            9       Medium
3  Sabarmati Industries Ltd           21       Medium
4      Om Logistics Pvt Ltd           36        Small

Columns before: 19, after dropping 2 columns: 17
Rows before adding case: 5000, after: 5001
Rows after removing demo row / rows with no date: 5000

=== 7. SORTING AND SHUFFLING ===
Top 5 by Authorized Capital (descending):
                          COMPANY_NAME  AUTHORIZED_CAP
2169  Aaryan Logistics Private Limited    3.990500e+10
2404      Aaryan Infra Private Limited    3.407500e+10
3926            Sabarmati Textiles Ltd    3.297400e+10
952      Gujarat Infra Private Limited    3.235900e+10
4544       Sabarmati Chemicals Pvt Ltd    3.229700e+10

Sorted by Class (asc) then Capital (desc):
                          COMPANY_NAME COMPANY_CLASS  AUTHORIZED_CAP
2169  Aaryan Logistics Private Limited       Private    3.990500e+10
2404      Aaryan Infra Private Limited       Private    3.407500e+10
3926            Sabarmati Textiles Ltd       Private    3.297400e+10
952      Gujarat Infra Private Limited       Private    3.235900e+10
4544       Sabarmati Chemicals Pvt Ltd       Private    3.229700e+10

After shuffling (first 5 names):
['Narmada Finserv Ltd', 'Narmada Industries Private Limited', 'Gujarat Chemicals
Private Limited', 'Aaryan Industries Ltd', 'Jay Chemicals Pvt Ltd']

Prepared data saved to gujarat_companies_prepared.csv  Shape: (5000, 17)
```

Signature of Faculty: **Grade:**

Practical –16

AIM: To perform data wrangling, identify and handle outliers, and apply data normalization and standardization techniques to the Company Master Data of Gujarat dataset.

- **Objectives:**
  - To understand data wrangling techniques.
  - To identify outliers in numerical attributes.
  - To apply suitable methods for handling outliers.
  - To perform data normalization and standardization.
  - To prepare numerical data for further analysis.

- **Theory:** Data wrangling is the process of cleaning, transforming and organizing data into a suitable format for analysis. Outliers are observations that differ significantly from the general pattern of the data and can be identified using techniques such as the Interquartile Range (IQR) method. Depending on the data and analysis requirements, outliers may be retained, removed or capped. Normalization rescales values to a specified range, commonly 0 to 1, while standardization transforms values to have a mean of approximately 0 and a standard deviation of approximately 1. These techniques help make numerical attributes more suitable for statistical analysis and machine learning.

- **Procedure:**

  1. Import the required Python libraries.
  2. Load the Company Master Data of Gujarat dataset.
  3. Identify the numerical attributes suitable for analysis.
  4. Detect outliers using an appropriate statistical technique.
  5. Apply a suitable method to handle the identified outliers.
  6. Perform data wrangling operations to clean and transform the data.
  7. Apply normalization and standardization to the selected numerical attributes.
  8. Compare the data before and after transformation.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, NumPy, SciPy and Scikit-learn libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler

pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)

# ------------------------------------------------------------ LOAD + WRANGLE
# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
df = df.drop_duplicates().reset_index(drop=True)

print("=== 1. DATA WRANGLING ===")
# type conversion
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
df["COMPANY_AGE"] = 2026 - df["REG_YEAR"]

# text cleaning
for col in ["COMPANY_NAME", "COMPANY_STATUS", "COMPANY_CLASS",
            "PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN"]:
    df[col] = df[col].astype(str).str.strip()
df["COMPANY_NAME"] = df["COMPANY_NAME"].str.title()

# missing values
print("Missing values before:\n", df[["AUTHORIZED_CAP", "PAIDUP_CAPITAL",
                                      "COMPANY_SUB_CATEGORY"]].isna().sum())
```

```
df["AUTHORIZED_CAP"] =
df["AUTHORIZED_CAP"].fillna(df["AUTHORIZED_CAP"].median())
# paid-up capital: fill with median paid-up ratio * authorized capital
ratio = (df["PAIDUP_CAPITAL"] / df["AUTHORIZED_CAP"]).median()
df["PAIDUP_CAPITAL"] = df["PAIDUP_CAPITAL"].fillna(df["AUTHORIZED_CAP"] *
ratio)
df["COMPANY_SUB_CATEGORY"] = df["COMPANY_SUB_CATEGORY"].fillna(
    df["COMPANY_SUB_CATEGORY"].mode()[0])
print("\nMissing values after:\n", df[["AUTHORIZED_CAP", "PAIDUP_CAPITAL",
                                      "COMPANY_SUB_CATEGORY"]].isna().sum())

# derived variable and binning
df["PAIDUP_RATIO"] = df["PAIDUP_CAPITAL"] / df["AUTHORIZED_CAP"]
df["CAPITAL_GROUP"] = pd.cut(df["AUTHORIZED_CAP"],
                             [0, 1e6, 1e7, 1e8, np.inf],
                             labels=["Small", "Medium", "Large", "Very
Large"])
print("\nCapital group counts:\n", df["CAPITAL_GROUP"].value_counts())

# ------------------------------------------------------------------ OUTLIERS
print("\n=== 2. OUTLIER DETECTION ===")
num_cols = ["AUTHORIZED_CAP", "PAIDUP_CAPITAL"]
for col in num_cols:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_iqr = ((df[col] < low) | (df[col] > high)).sum()
    z = np.abs(stats.zscore(df[col]))
    n_z = (z > 3).sum()
    print(f"{col}: IQR bounds = ({low:,.0f}, {high:,.0f}) -> {n_iqr} outliers
| "
          f"Z-score > 3 -> {n_z} outliers")

# boxplot before treatment
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
for a, col in zip(ax, num_cols):
    a.boxplot(df[col].dropna())
```

```
    a.set_yscale("log")
    a.set_title(f"{col} (before treatment, log scale)")
plt.tight_layout()
plt.savefig("outliers_boxplot.png", dpi=150)
plt.close()

print("\n=== 3. OUTLIER HANDLING ===")
clean = df.copy()
# Method A: removal using IQR
q1, q3 = df["AUTHORIZED_CAP"].quantile([0.25, 0.75])
iqr = q3 - q1
mask = df["AUTHORIZED_CAP"].between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
print(f"Method A (removal): rows {len(df)} -> {mask.sum()}")

# Method B: capping / winsorizing at 1st and 99th percentile
for col in num_cols:
    lo, hi = df[col].quantile([0.01, 0.99])
    clean[col + "_CAPPED"] = df[col].clip(lo, hi)
    print(f"Method B (capping) {col}: max {df[col].max():,.0f} -> "
          f"{clean[col + '_CAPPED'].max():,.0f}")

# Method C: log transformation (reduces skewness)
for col in num_cols:
    clean[col + "_LOG"] = np.log1p(df[col])
    print(f"Method C (log) {col}: skew {df[col].skew():.2f} -> "
          f"{clean[col + '_LOG'].skew():.2f}")

# ------------------------------------------------------------ NORMALIZATION
print("\n=== 4. NORMALIZATION / STANDARDIZATION ===")
x = clean["AUTHORIZED_CAP_CAPPED"]

# Manual formulas
clean["AUTH_MINMAX_MANUAL"] = (x - x.min()) / (x.max() - x.min())
clean["AUTH_ZSCORE_MANUAL"] = (x - x.mean()) / x.std(ddof=0)

# scikit-learn scalers
clean["AUTH_MINMAX"] = MinMaxScaler().fit_transform(x.to_frame())
```

```
clean["AUTH_STANDARD"] = StandardScaler().fit_transform(x.to_frame())
clean["AUTH_ROBUST"] = RobustScaler().fit_transform(x.to_frame())

# Decimal scaling
j = len(str(int(x.abs().max())))
clean["AUTH_DECIMAL"] = x / (10 ** j)

summary = clean[["AUTH_MINMAX", "AUTH_STANDARD", "AUTH_ROBUST",
"AUTH_DECIMAL"]].agg(
    ["min", "max", "mean", "std"]).round(4)
print(summary)
print("\nManual and sklearn Min-Max identical:",
      np.allclose(clean["AUTH_MINMAX"], clean["AUTH_MINMAX_MANUAL"]))
print("Manual and sklearn Z-score identical :",
      np.allclose(clean["AUTH_STANDARD"], clean["AUTH_ZSCORE_MANUAL"]))

# comparison plot
fig, ax = plt.subplots(1, 4, figsize=(15, 3.5))
for a, (col, title) in zip(ax, [("AUTHORIZED_CAP_CAPPED", "Capped (original
scale)"),
                                ("AUTH_MINMAX", "Min-Max [0,1]"),
                                ("AUTH_STANDARD", "Z-score"),
                                ("AUTH_ROBUST", "Robust")]):
    a.hist(clean[col], bins=40, color="steelblue")
    a.set_title(title)
plt.tight_layout()
plt.savefig("normalization_compare.png", dpi=150)
plt.close()

clean.to_csv("gujarat_companies_wrangled.csv", index=False)
print("\nSaved: gujarat_companies_wrangled.csv, outliers_boxplot.png, "
      "normalization_compare.png")
```

- **Output:**

```
python .\practical_16\prac16.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_16\prac16.py
=== 1. DATA WRANGLING ===
Missing values before:
 AUTHORIZED_CAP           50
PAIDUP_CAPITAL           100
COMPANY_SUB_CATEGORY     150
dtype: int64

Missing values after:
 AUTHORIZED_CAP            0
PAIDUP_CAPITAL           0
COMPANY_SUB_CATEGORY     0
dtype: int64

Capital group counts:
 CAPITAL_GROUP
Small          2945
Medium         1764
Large           264
Very Large       27
Name: count, dtype: int64

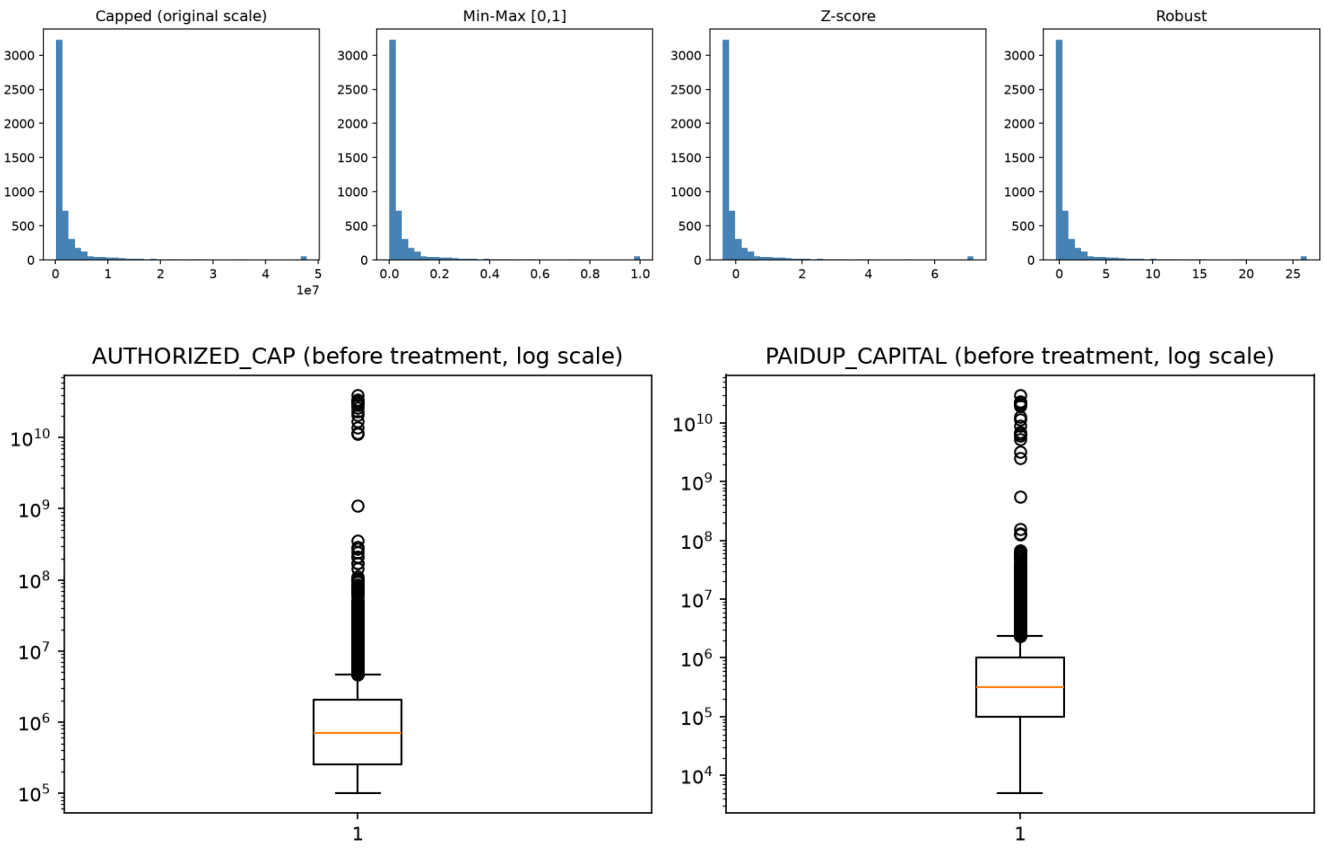
=== 2. OUTLIER DETECTION ===
AUTHORIZED_CAP: IQR bounds = (-2,420,000, 4,700,000) -> 587 outliers | Z-score > 3
-> 14 outliers
PAIDUP_CAPITAL: IQR bounds = (-1,276,375, 2,396,625) -> 590 outliers | Z-score > 3
-> 14 outliers

=== 3. OUTLIER HANDLING ===
Method A (removal): rows 5000 -> 4413
Method B (capping) AUTHORIZED_CAP: max 39,905,000,000 -> 47,864,700
Method B (capping) PAIDUP_CAPITAL: max 30,003,576,000 -> 26,135,090
Method C (log) AUTHORIZED_CAP: skew 21.44 -> 1.22
Method C (log) PAIDUP_CAPITAL: skew 25.22 -> 0.84

=== 4. NORMALIZATION / STANDARDIZATION ===
      AUTH_MINMAX  AUTH_STANDARD  AUTH_ROBUST  AUTH_DECIMAL
min        0.0000        -0.3968      -0.3371        0.0010
max        1.0000         7.1733      26.4970        0.4786
mean       0.0524         0.0000       1.0696        0.0260
std        0.1321         1.0001       3.5451        0.0631

Manual and sklearn Min-Max identical: True
Manual and sklearn Z-score identical : True

Saved: gujarat_companies_wrangled.csv, outliers_boxplot.png,
normalization_compare.png
```



Signature of Faculty: **Grade:**

Practical –17

AIM: To visualize the Company Master Data of Gujarat dataset using different graphs available in the Matplotlib library and compare their applicability.

- **Objectives:**
  - To understand different types of data visualization.
  - To create pie charts, bar charts, histograms, boxplots and scatterplots.
  - To visualize registration trends using line and area charts.
  - To interpret the information conveyed by different graphs.

- **Theory:** Data visualization represents information graphically, making patterns, trends and comparisons easier to understand. Matplotlib's `pyplot` module provides functions for creating different types of charts.
  - Pie and Bar Charts: Display proportions and comparisons among categories.
  - Histogram and Boxplot: Show frequency distributions and summarize numerical data, including potential outliers.
  - Scatterplot: Displays the relationship between two variables.
  - Line and Area Charts: Show changes over time, with area charts also emphasizing cumulative quantities.

  Selecting an appropriate graph depends on the data type and the objective of the analysis.

- **Procedure:**

1. Import Pandas and Matplotlib.
2. Load the Company Master Data of Gujarat dataset.
3. Create a pie chart showing companies by Company Status and Company Class.
4. Create a bar chart showing private/public companies by principal business activity.
5. Plot a histogram based on Registrar of Companies.
6. Create a boxplot for Authorized Capital.
7. Create a scatterplot for Authorized Capital and Registrar of Companies.
8. Plot the year-wise registration trend using a line chart.
9. Create an area chart showing the cumulative number of registered companies.
10. Compare the graphs and interpret their usefulness.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, Matplotlib and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import os
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt

SHOW = True
os.makedirs("output", exist_ok=True)

# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
df = df.drop_duplicates()
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
ACT = "PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN"


def finish(name):
    plt.tight_layout()
    plt.savefig(f"output/{name}.png", dpi=150)
    if SHOW:
        plt.show()
    plt.close()
```

```
# ------------------------------------------------------------ a) PIE CHART
fig, ax = plt.subplots(1, 2, figsize=(13, 6))
status = df["COMPANY_STATUS"].value_counts()
ax[0].pie(status, labels=status.index, autopct="%1.1f%%", startangle=90,
          textprops={"fontsize": 8})
ax[0].set_title("Percentage of companies by Company Status")
cls = df["COMPANY_CLASS"].value_counts()
ax[1].pie(cls, labels=cls.index, autopct="%1.1f%%", startangle=90,
          explode=[0.05] * len(cls), textprops={"fontsize": 8})
ax[1].set_title("Percentage of companies by Company Class")
finish("17a_pie_chart")

# ------------------------------------------------------------ b) BAR CHART
sub = df[df["COMPANY_CLASS"].isin(["Private", "Public"])]
pct = (pd.crosstab(sub[ACT], sub["COMPANY_CLASS"], normalize="index") * 100)
pct.plot(kind="barh", figsize=(11, 6), color=["#1f77b4", "#ff7f0e"])
plt.xlabel("Percentage of companies (%)")
plt.ylabel("Principal business activity")
plt.title("Private vs Public companies (%) by Principal Business Activity")
plt.legend(title="Class")
finish("17b_bar_chart")

# ------------------------------------------------------------ c) HISTOGRAM
# Registrar is categorical, so a count histogram (bar of counts) is drawn
roc = df["REGISTRAR_OF_COMPANIES"].value_counts()
plt.figure(figsize=(8, 5))
plt.bar(roc.index, roc.values, color="seagreen", edgecolor="black", width=1.0)
plt.xlabel("Registrar of Companies")
plt.ylabel("Number of companies")
plt.title("Companies with respect to Registrar of Companies")
finish("17c_histogram")

# ------------------------------------------------------------ d) BOXPLOT
plt.figure(figsize=(9, 5))
plt.boxplot(df["AUTHORIZED_CAP"].dropna(), showfliers=True)
plt.yscale("log")
```

```
plt.ylabel("Authorized capital (INR, log scale)")
plt.title("Boxplot of Authorized Capital")
plt.xticks([])
finish("17d_boxplot")

# ------------------------------------------------------------ e) SCATTERPLOT
plt.figure(figsize=(9, 5))
codes = df["REGISTRAR_OF_COMPANIES"].astype("category")
plt.scatter(codes.cat.codes + np.random.uniform(-0.15, 0.15, len(df)),
            df["AUTHORIZED_CAP"], s=8, alpha=0.4, c=codes.cat.codes,
cmap="tab10")
plt.xticks(range(len(codes.cat.categories)), codes.cat.categories)
plt.yscale("log")
plt.xlabel("Registrar of Companies (small jitter added)")
plt.ylabel("Authorized capital (INR, log scale)")
plt.title("Authorized Capital vs Registrar of Companies")
finish("17e_scatterplot")

# ------------------------------------------------------------------ f) LINE
yearly = df.groupby("REG_YEAR").size()
plt.figure(figsize=(10, 5))
plt.plot(yearly.index, yearly.values, marker="o", color="crimson")
plt.xlabel("Year of registration")
plt.ylabel("Number of companies registered")
plt.title("Year-wise trend of company registrations")
plt.grid(alpha=0.3)
finish("17f_line_chart")

# ------------------------------------------------------------------ g) AREA
cumulative = yearly.cumsum()
plt.figure(figsize=(10, 5))
plt.fill_between(cumulative.index, cumulative.values, alpha=0.5, color="teal")
plt.plot(cumulative.index, cumulative.values, color="teal")
plt.xlabel("Year")
plt.ylabel("Cumulative number of companies")
plt.title("Cumulative companies registered over the years")
finish("17g_area_chart")
```

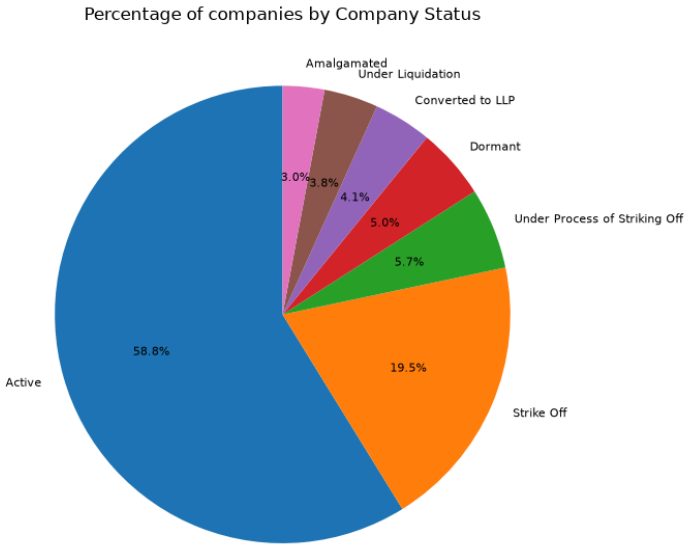
```
# ------------------------------------------------ COMPARISON (APPLICABILITY)
print("""
COMPARISON OF CHARTS WITH RESPECT TO APPLICABILITY
--------------------------------------------------
Pie chart   : Shows share of a whole; good for few categories (<=6). Hard to
read
              when there are many slices or similar-sized slices.
Bar chart   : Compares values across categories; works for many categories and
              for side-by-side comparison (private vs public).
Histogram   : Shows the frequency distribution of a variable (numeric data in
bins);
              here used as counts per registrar.
Boxplot     : Summarises spread (median, quartiles, outliers) of ONE numeric
              variable; best for spotting outliers such as huge authorized
capitals.
Scatterplot : Shows the relationship between two variables / groups; with
              categorical X it shows the spread of capital within each
registrar.
Line chart  : Best for time-series. The line connects ordered points so the
              trend (rise/fall) over years is easy to follow.
Area chart  : Like a line chart but the filled area emphasises magnitude;
ideal
              for cumulative totals over time.
""")
print("Line chart suits time series because time is continuous and ordered;
the "
      "connecting line shows direction, rate of change and seasonality that "
      "separate bars or points do not.")
print("\nAll figures saved in the 'output' folder.")
```

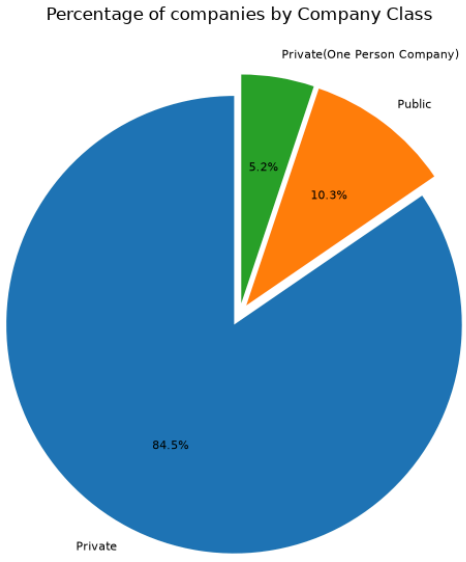
- **Output:**

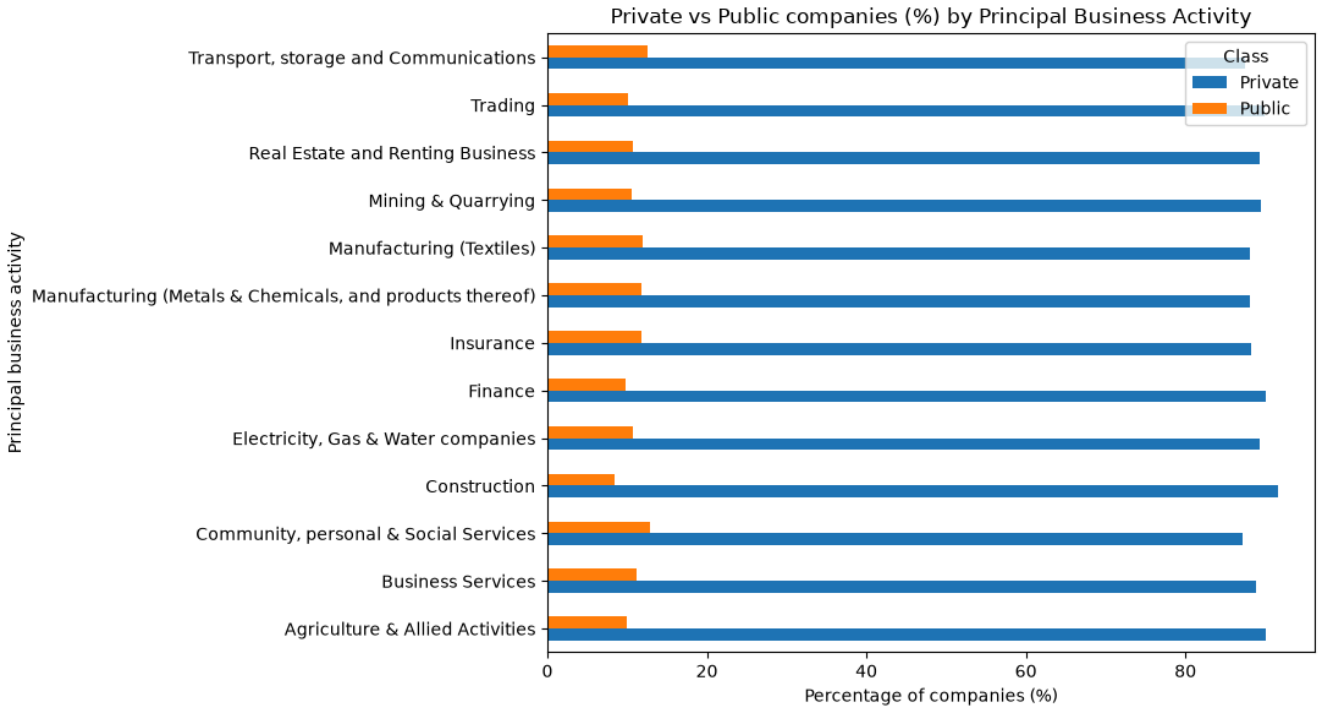
```
python .\practical_17\prac17.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_17\prac17.py

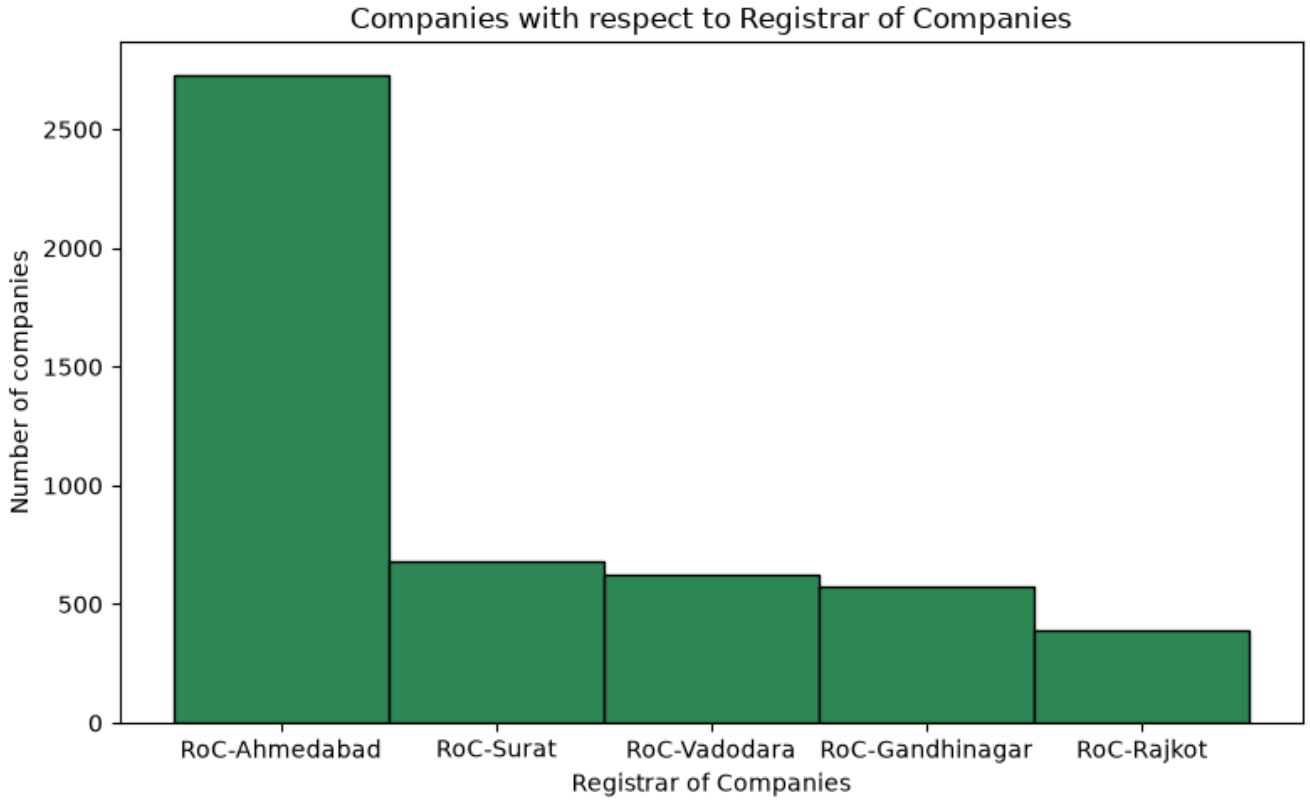
COMPARISON OF CHARTS WITH RESPECT TO APPLICABILITY
--------------------------------------------------
Pie chart   : Shows share of a whole; good for few categories (<=6). Hard to read
              when there are many slices or similar-sized slices.
Bar chart   : Compares values across categories; works for many categories and
              for side-by-side comparison (private vs public).
Histogram   : Shows the frequency distribution of a variable (numeric data in
bins);
              here used as counts per registrar.
Boxplot     : Summarises spread (median, quartiles, outliers) of ONE numeric
              variable; best for spotting outliers such as huge authorized
capitals.
Scatterplot : Shows the relationship between two variables / groups; with
              categorical X it shows the spread of capital within each registrar.
Line chart  : Best for time-series. The line connects ordered points so the
              trend (rise/fall) over years is easy to follow.
Area chart  : Like a line chart but the filled area emphasises magnitude; ideal
              for cumulative totals over time.

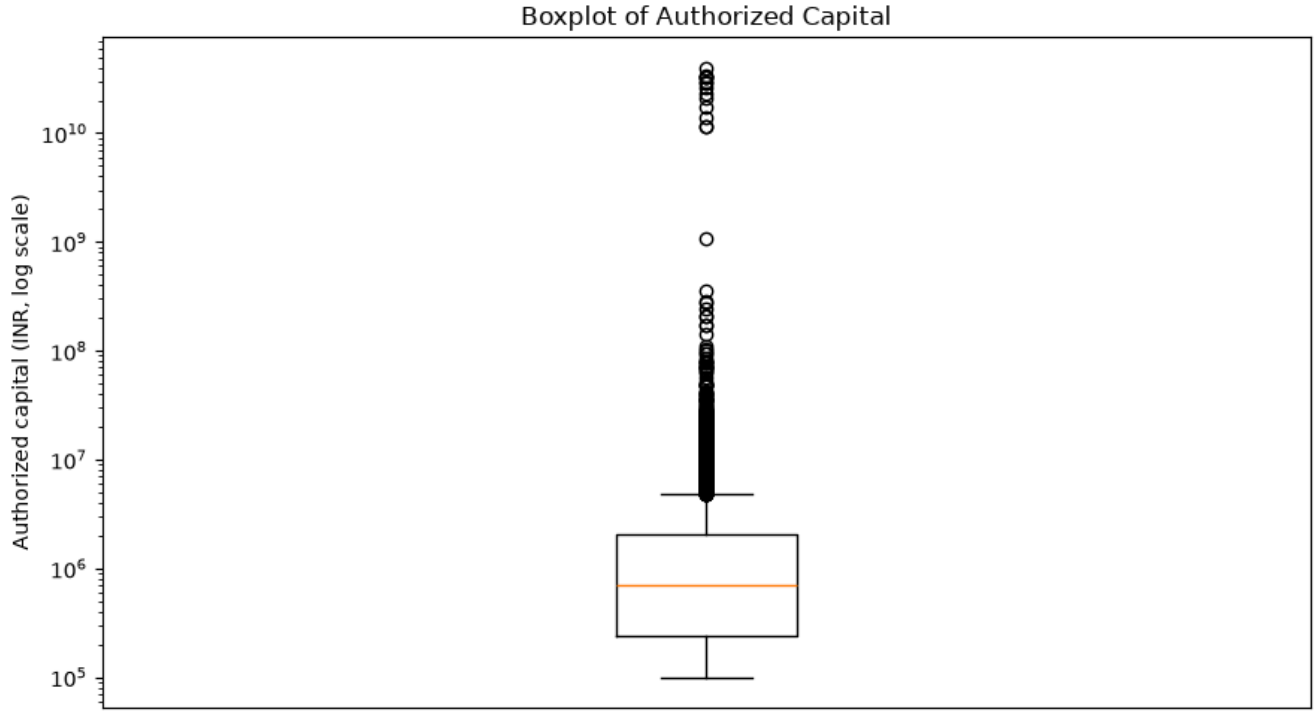
Line chart suits time series because time is continuous and ordered; the
connecting line shows direction, rate of change and seasonality that se
parate bars or points do not.
All figures saved in the 'output' folder.
```

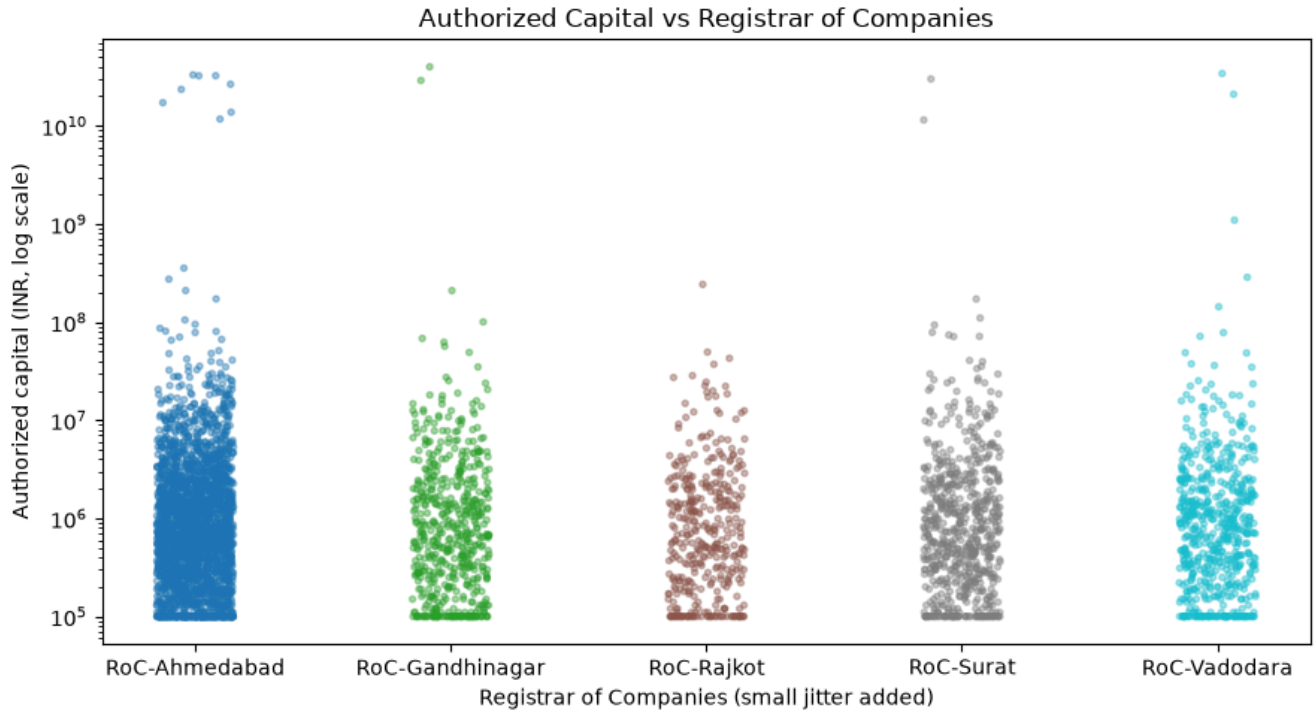




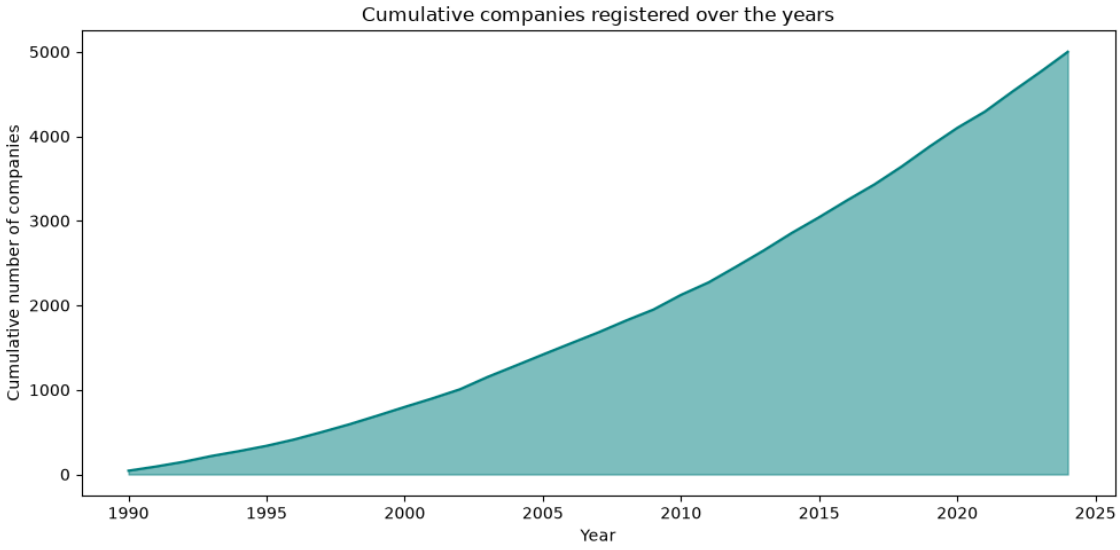

Private vs Public companies (%) by Principal Business Activity
Transport, storage and Communications
Trading
Real Estate and Renting Business
Mining & Quarrying
Manufacturing (Textiles)
Manufacturing (Metals & Chemicals, and products thereof)
Insurance
Finance
Electricity, Gas & Water companies
Construction
Community, personal & Social Services
Business Services
Agriculture & Allied Activities
Principal business activity
Class
Private
Public
0
20
40
60
80
Percentage of companies (%)


Companies with respect to Registrar of Companies
Number of companies
2500
2000
1500
1000
500
0
RoC-Ahmedabad
RoC-Surat
RoC-Vadodara
RoC-Gandhinagar
RoC-Rajkot
Registrar of Companies


Boxplot of Authorized Capital
Authorized capital (INR, log scale)
10^{10}
10^{9}
10^{8}
10^{7}
10^{6}
10^{5}


Authorized Capital vs Registrar of Companies
Authorized capital (INR, log scale)
10^{10}
10^{9}
10^{8}
10^{7}
10^{6}
10^{5}
RoC-Ahmedabad
RoC-Gandhinagar
RoC-Rajkot
RoC-Surat
RoC-Vadodara
Registrar of Companies (small jitter added)





Signature of Faculty: **Grade:**

Practical –18

AIM: To perform advanced data visualization using Matplotlib by creating an uneven multi-plot layout and a dual-axis chart, and saving the resulting figures in high-resolution formats.

- **Objectives:**
  - To understand advanced figure layout techniques.
  - To create multiple plots using GridSpec.
  - To construct a dual-axis chart using different scales.
  - To save figures in PNG and PDF formats at a resolution of at least 300 DPI.

- **Theory:** Advanced data visualization enables multiple datasets and metrics to be presented effectively within a single figure. Matplotlib's GridSpec allows the creation of flexible subplot layouts in which plots can occupy different numbers of rows and columns. A dual-axis chart uses two y-axes to display different metrics with different scales on the same figure, making their trends easier to compare. Figures can also be saved in formats such as PNG and PDF. A resolution of at least 300 DPI is commonly used for high-quality raster images, while PDF supports vector graphics for scalable output.

- **Procedure:**

  1. Import Pandas, NumPy and Matplotlib.
  2. Load and prepare the Company Master Data of Gujarat dataset.
  3. Select relevant attributes for visualization.
  4. Create a figure with an uneven subplot layout using GridSpec.
  5. Plot different attributes in the respective subplots.
  6. Create a dual-axis chart comparing two metrics with different scales.
  7. Add appropriate titles, axis labels, legends and formatting.
  8. Save the final figure in PNG and PDF formats with a resolution of at least 300 DPI.
  9. Examine the saved figures and interpret the visualized information.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, Matplotlib and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import os
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

SHOW = True
os.makedirs("output", exist_ok=True)

# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
df = df.drop_duplicates()
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
ACT = "PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN"

# ------------------------------------------------ a) GRIDSPEC UNEVEN LAYOUT
fig = plt.figure(figsize=(15, 10))
gs = GridSpec(3, 3, figure=fig, height_ratios=[1.2, 1, 1], width_ratios=[2, 1,
1])

# big plot spanning the whole top row
ax1 = fig.add_subplot(gs[0, :])
yearly = df.groupby("REG_YEAR").size()
ax1.plot(yearly.index, yearly.values, marker="o", color="crimson")
ax1.fill_between(yearly.index, yearly.values, alpha=0.2, color="crimson")
ax1.set_title("Companies registered per year")
ax1.set_xlabel("Year")
ax1.set_ylabel("Count")
```

```
# tall plot on the left spanning two rows
ax2 = fig.add_subplot(gs[1:, 0])
act = df[ACT].value_counts().sort_values()
ax2.barh(act.index, act.values, color="steelblue")
ax2.set_title("Companies by principal business activity")
ax2.tick_params(axis="y", labelsize=7)

# small plots on the right
ax3 = fig.add_subplot(gs[1, 1])
status = df["COMPANY_STATUS"].value_counts()
ax3.pie(status, labels=status.index, autopct="%1.0f%%", textprops={"fontsize":
6})
ax3.set_title("Company status")

ax4 = fig.add_subplot(gs[1, 2])
cls = df["COMPANY_CLASS"].value_counts()
ax4.bar(range(len(cls)), cls.values, color=["#4c72b0", "#dd8452", "#55a868"])
ax4.set_xticks(range(len(cls)))
ax4.set_xticklabels(["Private", "Public", "OPC"][:len(cls)] if len(cls) == 3
else cls.index,
                    fontsize=8)
ax4.set_title("Company class")

ax5 = fig.add_subplot(gs[2, 1:])
ax5.hist(np.log10(df["AUTHORIZED_CAP"].dropna()), bins=40, color="seagreen",
         edgecolor="white")
ax5.set_title("Distribution of authorized capital (log10 INR)")
ax5.set_xlabel("log10(Authorized capital)")

fig.suptitle("Company Master Data of Gujarat - Overview Dashboard",
fontsize=16)
fig.tight_layout(rect=[0, 0, 1, 0.96])

# c) SAVE AT HIGH RESOLUTION
fig.savefig("output/gridspec_dashboard.png", dpi=300)
fig.savefig("output/gridspec_dashboard.pdf", dpi=300)
if SHOW:
```

```
    plt.show()
plt.close(fig)

# ------------------------------------------------- b) DUAL-AXIS (TWIN Y)
per_year = df.groupby("REG_YEAR").agg(companies=("COMPANY_NAME", "count"),
                                      avg_cap=("AUTHORIZED_CAP", "mean"))
fig, ax_left = plt.subplots(figsize=(11, 6))
ax_left.bar(per_year.index, per_year["companies"], color="lightsteelblue",
            label="Companies registered")
ax_left.set_xlabel("Year of registration")
ax_left.set_ylabel("Number of companies", color="steelblue")
ax_left.tick_params(axis="y", labelcolor="steelblue")

ax_right = ax_left.twinx()                          # second y-axis sharing the
x-axis
ax_right.plot(per_year.index, per_year["avg_cap"] / 1e6, color="darkred",
              marker="o", label="Average authorized capital")
ax_right.set_ylabel("Average authorized capital (INR million)",
color="darkred")
ax_right.tick_params(axis="y", labelcolor="darkred")

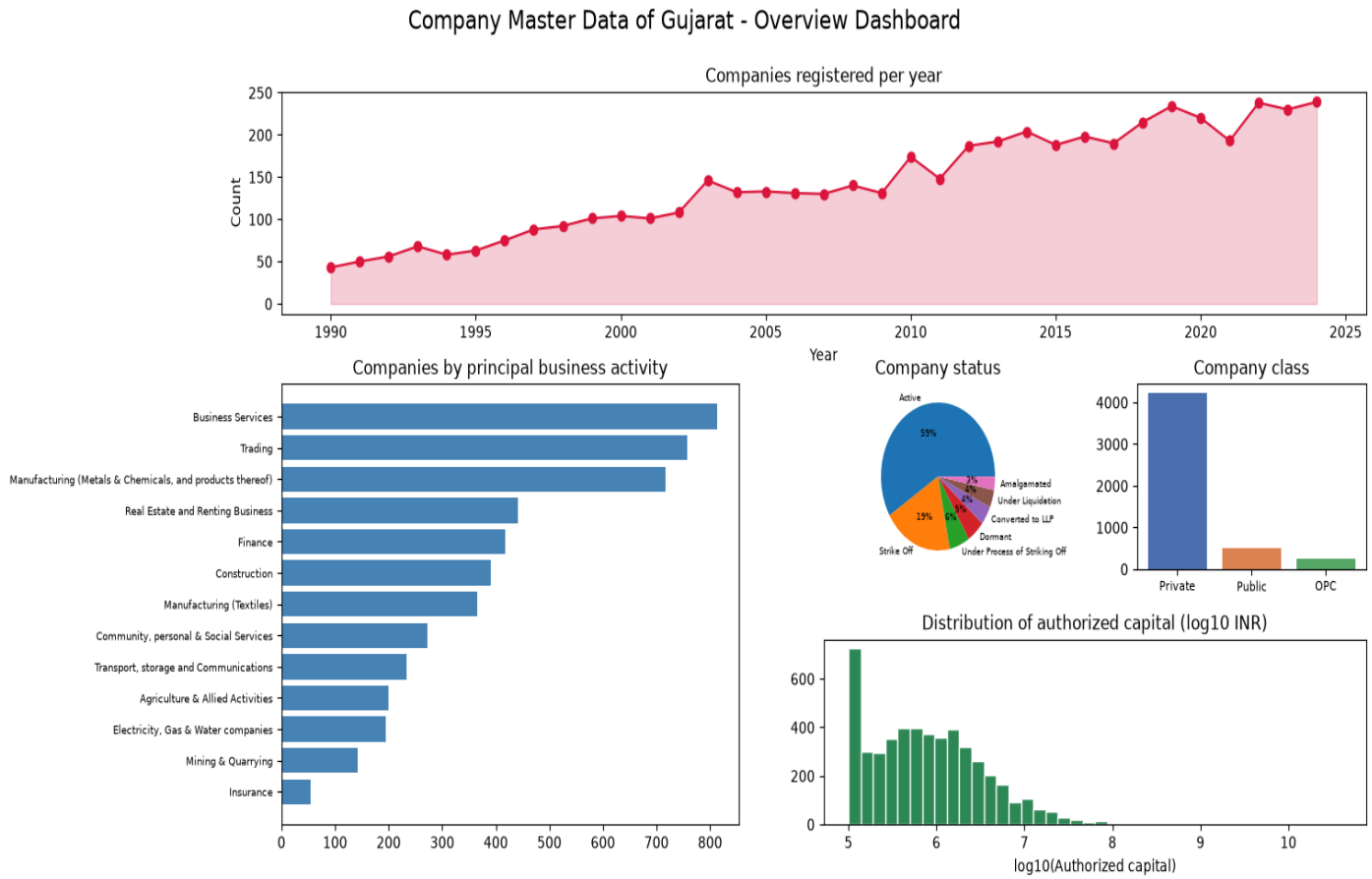
lines = ax_left.get_legend_handles_labels()
lines2 = ax_right.get_legend_handles_labels()
ax_left.legend(lines[0] + lines2[0], lines[1] + lines2[1], loc="upper left")
plt.title("Companies registered vs average authorized capital per year")
fig.tight_layout()

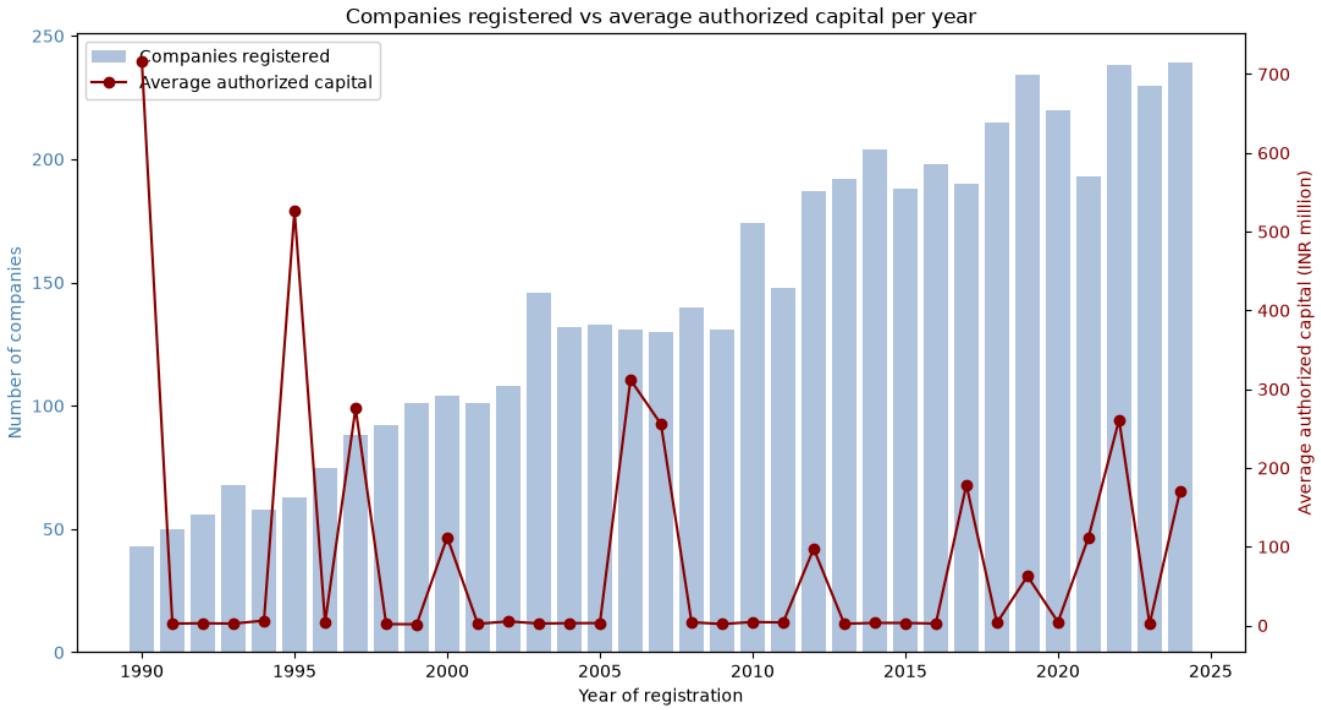
fig.savefig("output/dual_axis_chart.png", dpi=300)
fig.savefig("output/dual_axis_chart.pdf", dpi=300)
if SHOW:
    plt.show()
plt.close(fig)

print("Saved (300 dpi): output/gridspec_dashboard.png/.pdf and "
      "output/dual_axis_chart.png/.pdf")
```

- **Output:**

```
python .\practical_18\prac18.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_18\prac18.py
Saved (300 dpi): output/gridspec_dashboard.png/.pdf and
output/dual_axis_chart.png/.pdf
```





Signature of Faculty: **Grade:**

Practical –19

AIM: To perform statistical data visualization on the Company Master Data of Gujarat dataset using different plotting techniques available in the Seaborn library.

- **Objectives:**
  - To understand statistical visualization using Seaborn.
  - To create bar plots, strip plots, violin plots, pair plots and KDE plots.
  - To visualize categorical and numerical relationships.
  - To interpret distributions and relationships among dataset attributes.

- **Theory:** Seaborn is a Python data visualization library built on Matplotlib that provides functions for creating informative statistical graphics.
  - **Bar Plot:** Compares a numerical measure across categories.
  - **Strip Plot:** Displays individual observations across categories.
  - **Violin Plot:** Shows the distribution and density of numerical values across categories.
  - **Pair Plot:** Displays pairwise relationships among multiple numerical attributes.
  - **Kernel Density Estimation (KDE) Plot:** Provides a smoothed representation of a variable's probability density.

  These visualizations help compare categories, understand data distributions and identify relationships between attributes.

- **Procedure:**

1. Import Pandas, Matplotlib and Seaborn.
2. Load the Company Master Data of Gujarat dataset.
3. Prepare the relevant categorical and numerical attributes.
4. Create a bar plot for private/public companies by principal business activity.
5. Create a strip plot for Authorized Capital across Company Classes.
6. Create a violin plot showing Authorized Capital distributions across Company Classes.
7. Generate a pair plot for selected numerical attributes.
8. Create a KDE plot of Authorized Capital across suitable categories.
9. Add appropriate labels and titles, and interpret the resulting graphs.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, Matplotlib, Seaborn and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import os
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

SHOW = True
os.makedirs("output", exist_ok=True)
sns.set_theme(style="whitegrid")

# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
df = df.drop_duplicates()
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
df = df.dropna(subset=["AUTHORIZED_CAP", "PAIDUP_CAPITAL"])
df["LOG_AUTH_CAP"] = np.log10(df["AUTHORIZED_CAP"])
df["LOG_PAIDUP_CAP"] = np.log10(df["PAIDUP_CAPITAL"].clip(lower=1))
ACT = "PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN"


def finish(name):
    plt.tight_layout()
```

```
    plt.savefig(f"output/{name}.png", dpi=150)
    if SHOW:
        plt.show()
    plt.close()


# ------------------------------------------------------------ a) BAR PLOT
sub = df[df["COMPANY_CLASS"].isin(["Private", "Public"])]
pct = (sub.groupby(ACT)["COMPANY_CLASS"].value_counts(normalize=True)
       .mul(100).rename("PERCENT").reset_index())
plt.figure(figsize=(11, 6))
sns.barplot(data=pct, y=ACT, x="PERCENT", hue="COMPANY_CLASS", palette="Set2")
plt.xlabel("Percentage of companies (%)")
plt.ylabel("Principal business activity")
plt.title("Private vs Public companies (%) by Principal Business Activity")
finish("19a_barplot")

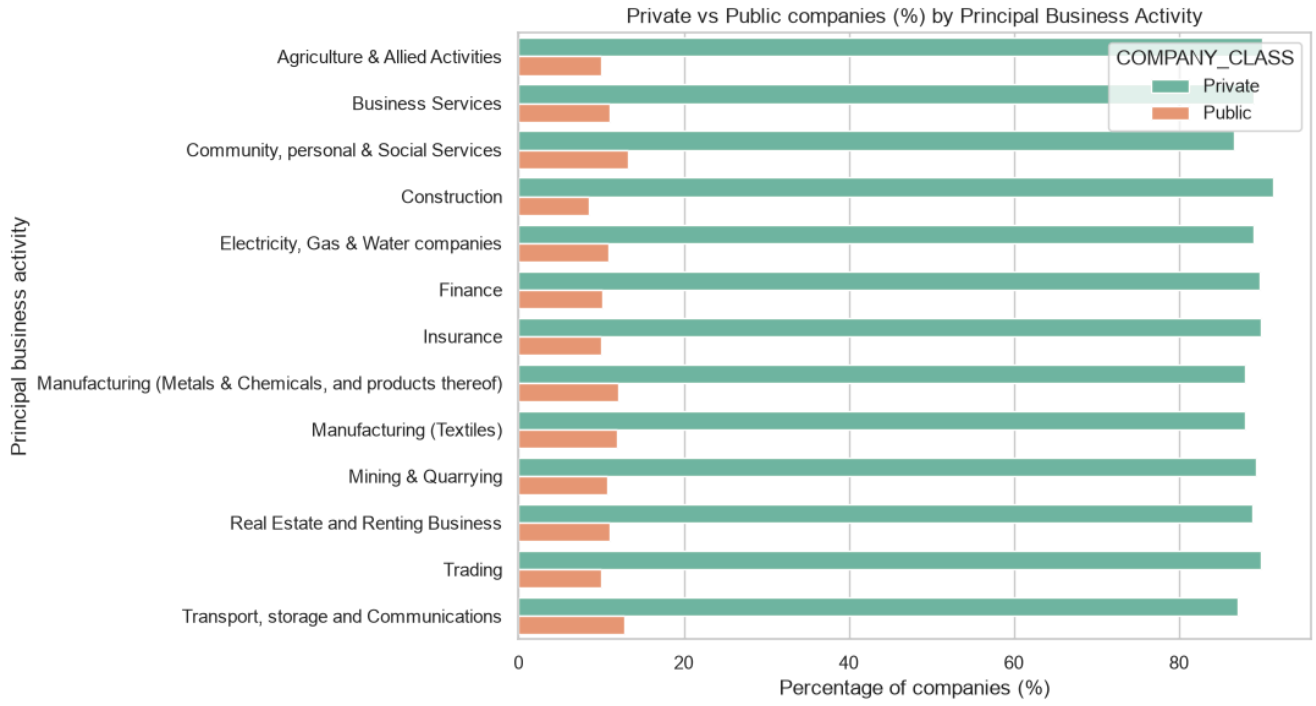
# ------------------------------------------------------------ b) STRIP PLOT
plt.figure(figsize=(9, 6))
sample = df.sample(1500, random_state=1)            # sample keeps the plot
readable
sns.stripplot(data=sample, x="COMPANY_CLASS", y="AUTHORIZED_CAP", jitter=0.25,
              alpha=0.5, size=3, palette="Set1", hue="COMPANY_CLASS",
legend=False)
plt.yscale("log")
plt.xlabel("Company class")
plt.ylabel("Authorized capital (INR, log scale)")
plt.title("Strip plot: Authorized Capital by Company Class (1500-company
sample)")
finish("19b_stripplot")
# ------------------------------------------------------------ c) VIOLIN PLOT
plt.figure(figsize=(9, 6))
sns.violinplot(data=df, x="COMPANY_CLASS", y="LOG_AUTH_CAP", inner="quartile",
               palette="muted", hue="COMPANY_CLASS", legend=False)
plt.xlabel("Company class")
plt.ylabel("log10(Authorized capital)")
plt.title("Violin plot: Distribution of Authorized Capital across Company
```

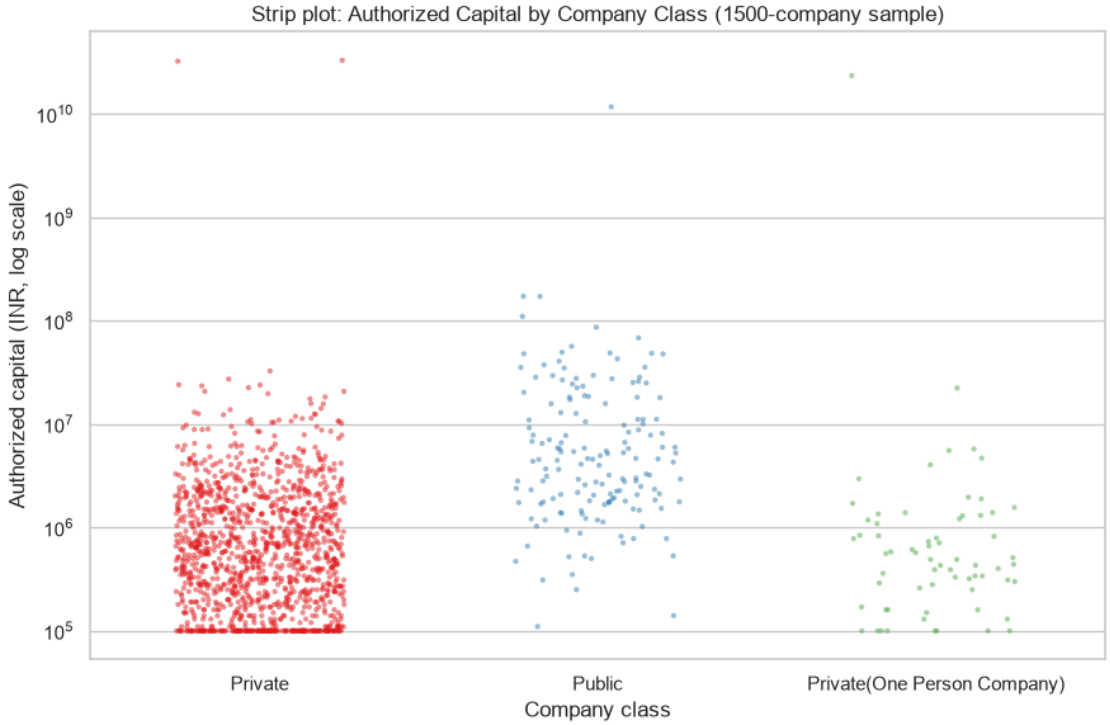
```
Classes")
finish("19c_violinplot")
# ------------------------------------------------------------ d) PAIR PLOT
pair_cols = ["LOG_AUTH_CAP", "LOG_PAIDUP_CAP", "REG_YEAR"]
g = sns.pairplot(df.sample(1500, random_state=2)[pair_cols +
["COMPANY_CLASS"]],
                 hue="COMPANY_CLASS", diag_kind="kde",
                 plot_kws={"alpha": 0.5, "s": 15})
g.figure.suptitle("Pair plot of numeric attributes", y=1.02)
g.savefig("output/19d_pairplot.png", dpi=150)
if SHOW:
    plt.show()
plt.close("all")
# ------------------------------------------------------------ e) KDE PLOT
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
sns.kdeplot(data=df, x="LOG_AUTH_CAP", hue="COMPANY_CLASS", fill=True,
            common_norm=False, alpha=0.4, ax=ax[0])
ax[0].set_title("KDE of Authorized Capital by Company Class")
ax[0].set_xlabel("log10(Authorized capital)")
sns.kdeplot(data=df, x="LOG_AUTH_CAP", hue="COMPANY_STATUS",
common_norm=False,
            ax=ax[1], legend=True)
ax[1].set_title("KDE of Authorized Capital by Company Status")
ax[1].set_xlabel("log10(Authorized capital)")
finish("19e_kdeplot")
print("""
INTERPRETATION NOTES
- Bar plot   : compares percentage of private vs public companies per
activity.
- Strip plot : each dot is a company; shows individual values and density.
- Violin plot: combines a boxplot (quartiles) with a KDE to show the shape.
- Pair plot  : scatter plots of every pair plus distribution on the diagonal.
- KDE plot   : smooth estimate of the probability density of authorized
capital.
(Capital is plotted on a log scale because it is highly right-skewed.)
All figures saved in the 'output' folder.""")
```

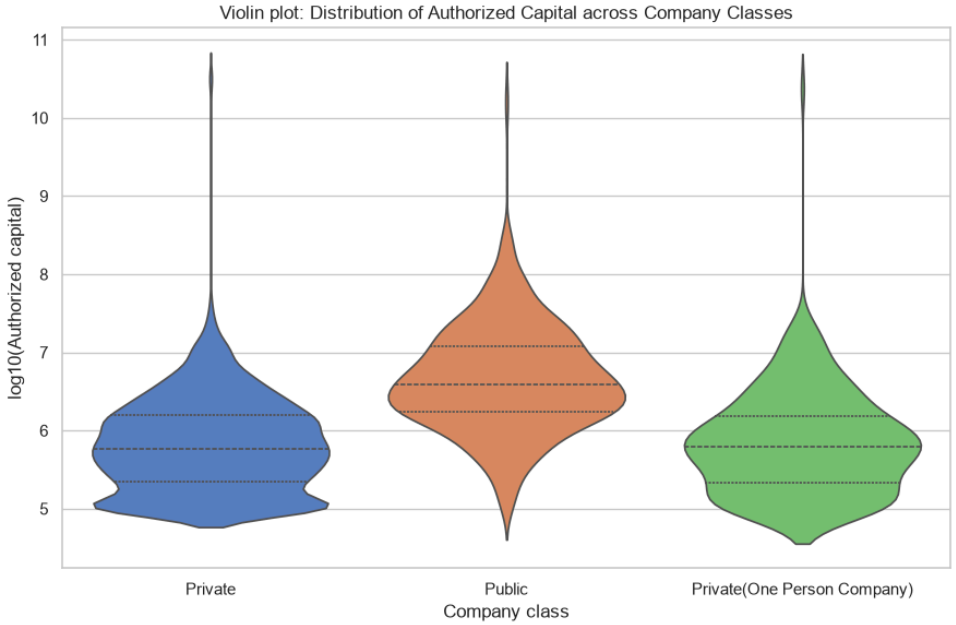
- **Output:**

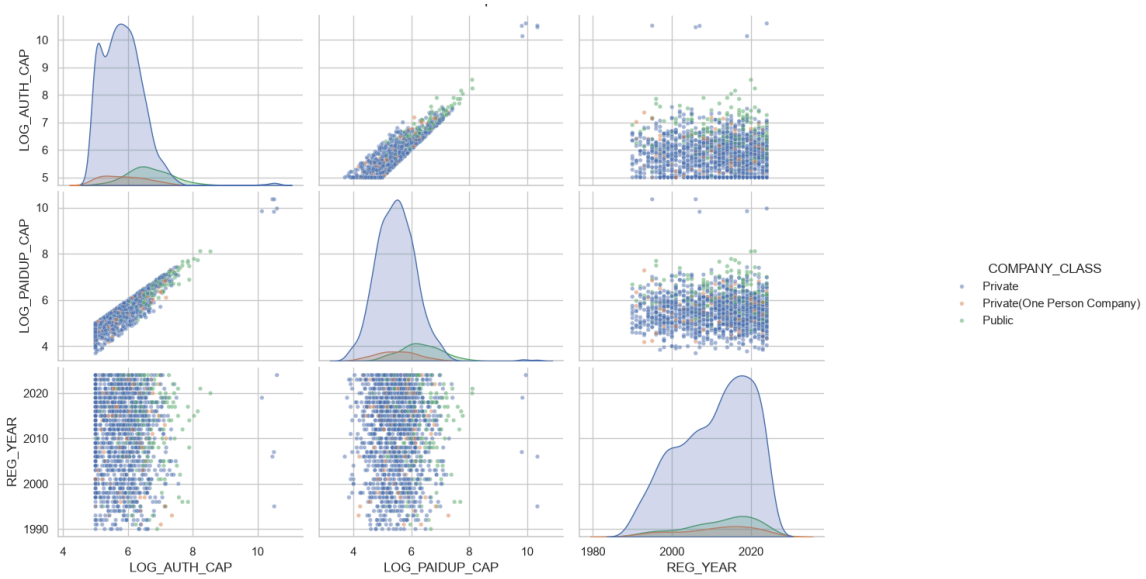
```
python .\practical_19\prac19.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_19\prac19.py

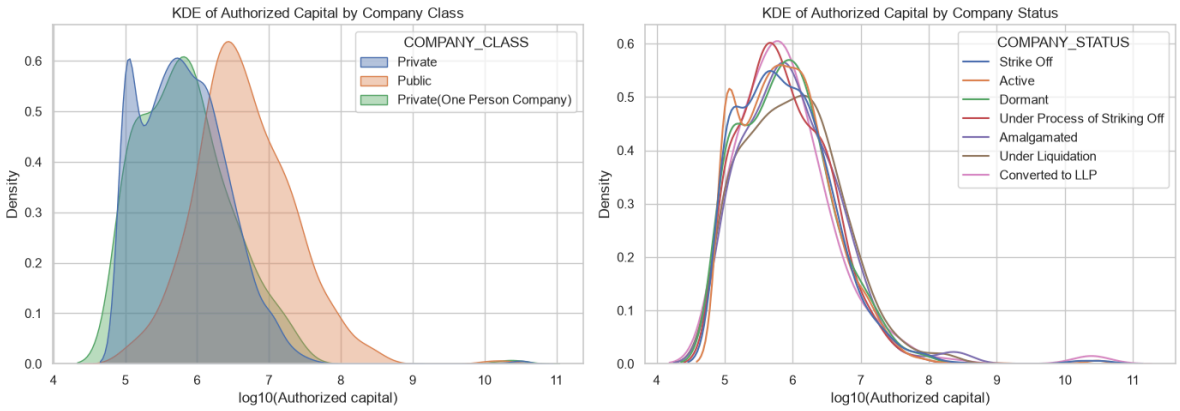
INTERPRETATION NOTES
- Bar plot   : compares percentage of private vs public companies
per activity.
- Strip plot : each dot is a company; shows individual values and
density.
- Violin plot: combines a boxplot (quartiles) with a KDE to show the
shape.
- Pair plot  : scatter plots of every pair plus distribution on the
diagonal.
- KDE plot   : smooth estimate of the probability density of
authorized capital.
(Capital is plotted on a log scale because it is highly
right-skewed.)
All figures saved in the 'output' folder.
```




Strip plot: Authorized Capital by Company Class (1500-company sample)
Authorized capital (INR, log scale)
10^{10}
10^{9}
10^{8}
10^{7}
10^{6}
10^{5}
Private
Public
Private(One Person Company)
Company class


Violin plot: Distribution of Authorized Capital across Company Classes
log10(Authorized capital)
11
10
9
8
7
6
5
Private
Public
Private(One Person Company)
Company class





Signature of Faculty: **Grade:**

Practical –20

AIM: To perform correlation analysis on the Company Master Data of Gujarat dataset by calculating covariance and Pearson correlation matrices, visualizing correlations using a heatmap, and conducting chi-square tests of independence on categorical variables.

- **Objectives:**
  - To understand covariance and correlation between numerical variables.
  - To calculate covariance and Pearson correlation matrices.
  - To visualize relationships using a correlation heatmap.
  - To perform chi-square tests of independence on categorical variables.
  - To interpret statistical results and identify relationships among attributes.

- **Theory:** Correlation analysis is used to study the strength and direction of relationships between variables.
  - Covariance: Indicates whether two numerical variables tend to increase or decrease together. Its magnitude depends on the scales of the variables.
  - Pearson Correlation: Measures the strength and direction of the linear relationship between two numerical variables. Its coefficient ranges from −1 to +1.
  - Heatmap: Represents correlation coefficients using colors, allowing positive, negative and weak relationships to be identified visually.
  - Chi-Square Test of Independence: Determines whether two categorical variables are statistically associated. The test compares observed frequencies with the frequencies expected under the assumption of independence. The p-value is used to assess statistical significance.

  Correlation indicates association, not necessarily causation.

- **Procedure:**

  1. Import the required data analysis, statistical and visualization libraries.
  2. Load the Company Master Data of Gujarat dataset.
  3. Select the relevant numerical attributes.
  4. Calculate the covariance matrix.
  5. Calculate the Pearson correlation matrix.
  6. Visualize the correlation matrix using a heatmap.
  7. Select suitable categorical variables and construct contingency tables.
  8. Perform chi-square tests of independence on the selected categorical variables.
  9. Interpret the correlation coefficients, heatmap and chi-square test results.

- **Tools / Material Needed:**
  - **Hardware:** Computer or laptop, Minimum 4 GB RAM.
  - **Software:** Python 3.x., Jupyter Notebook or VS Code, Pandas, SciPy, Matplotlib, Seaborn and NumPy libraries.
  - **Dataset:** Company Master Data of Gujarat Dataset

- **Code:**

```
import os
import numpy as np
from pathlib import Path
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import chi2_contingency
SHOW = True
os.makedirs("output", exist_ok=True)
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 20)
# single shared dataset: <project>/data/gujarat_companies.csv
DATA_PATH = Path(__file__).resolve().parent.parent / "data" /
"gujarat_companies.csv"
df = pd.read_csv(DATA_PATH)
df.columns = df.columns.str.strip().str.upper()
df = df.drop_duplicates()
```

```
df["DATE_OF_REGISTRATION"] = pd.to_datetime(df["DATE_OF_REGISTRATION"],
                                            format="%d-%m-%Y",
errors="coerce")
df["REG_YEAR"] = df["DATE_OF_REGISTRATION"].dt.year
df["COMPANY_AGE"] = 2026 - df["REG_YEAR"]
df = df.dropna(subset=["AUTHORIZED_CAP", "PAIDUP_CAPITAL", "REG_YEAR"])
df["PAIDUP_RATIO"] = df["PAIDUP_CAPITAL"] / df["AUTHORIZED_CAP"]
num_cols = ["AUTHORIZED_CAP", "PAIDUP_CAPITAL", "REG_YEAR", "COMPANY_AGE",
"PAIDUP_RATIO"]
num = df[num_cols]
# ------------------------------------------------ a) COVARIANCE AND PEARSON
print("=== a) COVARIANCE MATRIX ===")
print(num.cov())
print("\n=== PEARSON CORRELATION MATRIX ===")
corr = num.corr(method="pearson")
print(corr.round(3))
print("\nInterpretation:")
print("- Correlation lies between -1 and +1; covariance depends on the
units.")
print(f"- AUTHORIZED_CAP vs PAIDUP_CAPITAL r = {corr.loc['AUTHORIZED_CAP',
'PAIDUP_CAPITAL']:.3f} "
      "(paid-up capital can never exceed authorized capital).")
print(f"- REG_YEAR vs COMPANY_AGE r = {corr.loc['REG_YEAR',
'COMPANY_AGE']:.3f} "
      "(perfect negative: age is derived from year).")
# ------------------------------------------------------------- b) HEATMAP
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", vmin=-1, vmax=1,
            linewidths=0.5, square=True)
plt.title("Pearson correlation heatmap")
plt.tight_layout()
plt.savefig("output/correlation_heatmap.png", dpi=150)
if SHOW:
    plt.show()
plt.close()
# ------------------------------------------------------- c) CHI-SQUARE TESTS
def chi_square(col1, col2, alpha=0.05):
```

```
    table = pd.crosstab(df[col1], df[col2])
    chi2, p, dof, expected = chi2_contingency(table)
    print(f"\n--- {col1}  vs  {col2} ---")
    print(table)
    print(f"chi2 = {chi2:.3f}, degrees of freedom = {dof}, p-value = {p:.4f}")
    if (expected < 5).any():
        print("Note: some expected counts < 5, so the result is approximate.")
    if p < alpha:
        print(f"Result: p < {alpha} -> reject H0: the variables are DEPENDENT
(associated).")
    else:
        print(f"Result: p >= {alpha} -> fail to reject H0: no evidence of
association "
              "(variables independent).")
print("\n=== c) CHI-SQUARE TEST FOR INDEPENDENCE ===")
print("H0: the two categorical variables are independent. H1: they are
associated.")
chi_square("COMPANY_CLASS", "COMPANY_STATUS")
chi_square("COMPANY_CLASS", "REGISTRAR_OF_COMPANIES")
chi_square("COMPANY_STATUS", "PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN")
print("\nAll figures saved in the 'output' folder.")
```

- **Output:**

```
python .\practical_20\prac20.py
(venv) PS E:\College\124_5TH_SEM\PDS\124_pds_practicals> python
.\practical_20\prac20.py
=== a) COVARIANCE MATRIX ===
                AUTHORIZED_CAP  PAIDUP_CAPITAL      REG_YEAR   COMPANY_AGE
PAIDUP_RATIO
AUTHORIZED_CAP    2.052640e+18    1.112383e+18 -2.917998e+07  2.917998e+07
2.607071e+06
PAIDUP_CAPITAL    1.112383e+18    7.764425e+17  7.493533e+06 -7.493533e+06
7.407289e+06
REG_YEAR         -2.917998e+07    7.493533e+06  8.437390e+01 -8.437390e+01
-1.051816e-02
COMPANY_AGE       2.917998e+07   -7.493533e+06 -8.437390e+01  8.437390e+01
1.051816e-02
PAIDUP_RATIO      2.607071e+06    7.407289e+06 -1.051816e-02  1.051816e-02
7.381854e-02
```

```
=== PEARSON CORRELATION MATRIX ===
                AUTHORIZED_CAP  PAIDUP_CAPITAL  REG_YEAR  COMPANY_AGE
PAIDUP_RATIO
AUTHORIZED_CAP           1.000           0.881    -0.002        0.002
0.007
PAIDUP_CAPITAL           0.881           1.000     0.001       -0.001
0.031
REG_YEAR                -0.002           0.001     1.000       -1.000
-0.004
COMPANY_AGE              0.002          -0.001    -1.000        1.000
0.004
PAIDUP_RATIO             0.007           0.031    -0.004        0.004
1.000

Interpretation:
- Correlation lies between -1 and +1; covariance depends on the units.
- AUTHORIZED_CAP vs PAIDUP_CAPITAL r = 0.881 (paid-up capital can never exceed
authorized capital).
- REG_YEAR vs COMPANY_AGE r = -1.000 (perfect negative: age is derived from year).

=== c) CHI-SQUARE TEST FOR INDEPENDENCE ===
H0: the two categorical variables are independent. H1: they are associated.

--- COMPANY_CLASS  vs  COMPANY_STATUS ---
COMPANY_STATUS               Active  Amalgamated  Converted to LLP  Dormant
Strike Off  Under Liquidation  Under Process of Striking Off
COMPANY_CLASS
Private                        2441          118               163      217
774                 154                              237
Private(One Person Company)     143            7                10        7
46                  12                               17
Public                          266           19                24       24
128                  19                               25
chi2 = 19.977, degrees of freedom = 12, p-value = 0.0675
Result: p >= 0.05 -> fail to reject H0: no evidence of association (variables
independent).

--- COMPANY_CLASS  vs  REGISTRAR_OF_COMPANIES ---
REGISTRAR_OF_COMPANIES       RoC-Ahmedabad  RoC-Gandhinagar  RoC-Rajkot  RoC-Surat
RoC-Vadodara
COMPANY_CLASS
Private                               2242              477         320        550
515
Private(One Person Company)            147               23          19         25
28
Public                                 260               60          37         82
66
chi2 = 8.552, degrees of freedom = 8, p-value = 0.3815
Result: p >= 0.05 -> fail to reject H0: no evidence of association (variables
independent).

--- COMPANY_STATUS  vs  PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN ---
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  Agriculture & Allied Activities  Business
```

```
Services  Community, personal & Social Services  Construction
 Electricity, Gas & Water companies  Finance  \
COMPANY_STATUS

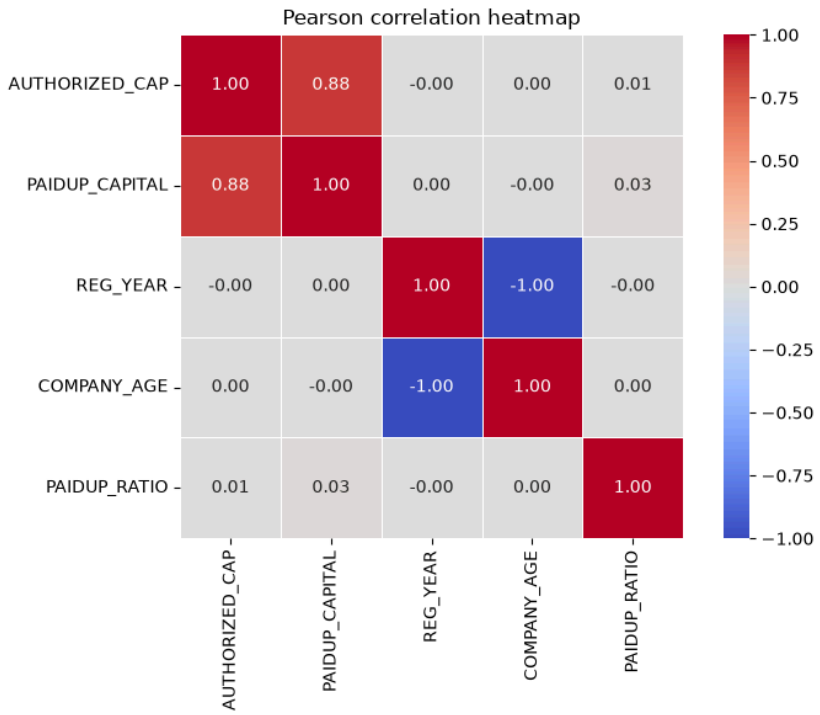
Active                                                                  122
471                                     148           219
                                  117      244
Amalgamated                                                               4
20                                       8            14
                                   11        7
Converted to LLP                                                          7
41                                      11            13
                                    2       23
Dormant                                                                   8
35                                      12            17
                                    5       21
Strike Off                                                               35
145                                      51            74
                                   39       71
Under Liquidation                                                         6
29                                       6            16
                                    6       16
Under Process of Striking Off                                            12
47                                      20            27
                                    9       22
PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  Insurance  Manufacturing (Metals &
Chemicals, and products thereof)  Manufacturing (Textiles)  Mining &
Quarrying  Real Estate and Renting Business  Trading  \
COMPANY_STATUS

Active                                              34
398                                196
       82                                  250      433
Amalgamated                                          2
29                                 12
        3                                    8       15
Converted to LLP                                     1
28                                 20
        3                                   19       22
Dormant                                              5
42                                 23
        8                                   22       35
Strike Off                                          10
135                                 73
       31                                   88      157
Under Liquidation                                    2
27                                  9
        2                                   13       40
Under Process of Striking Off                        0
37                                 23
       12                                   24       40

PRINCIPAL_BUSINESS_ACTIVITY_AS_PER_CIN  Transport, storage and Communications
```

```
COMPANY_STATUS
Active                                                                  136
Amalgamated                                                              11
Converted to LLP                                                          7
Dormant                                                                  15
Strike Off                                                               39
Under Liquidation                                                        13
Under Process of Striking Off                                             6
chi2 = 79.247, degrees of freedom = 72, p-value = 0.2611
Note: some expected counts < 5, so the result is approximate.
Result: p >= 0.05 -> fail to reject H0: no evidence of association (variables
independent).

All figures saved in the 'output' folder.
```



Signature of Faculty: **Grade:**